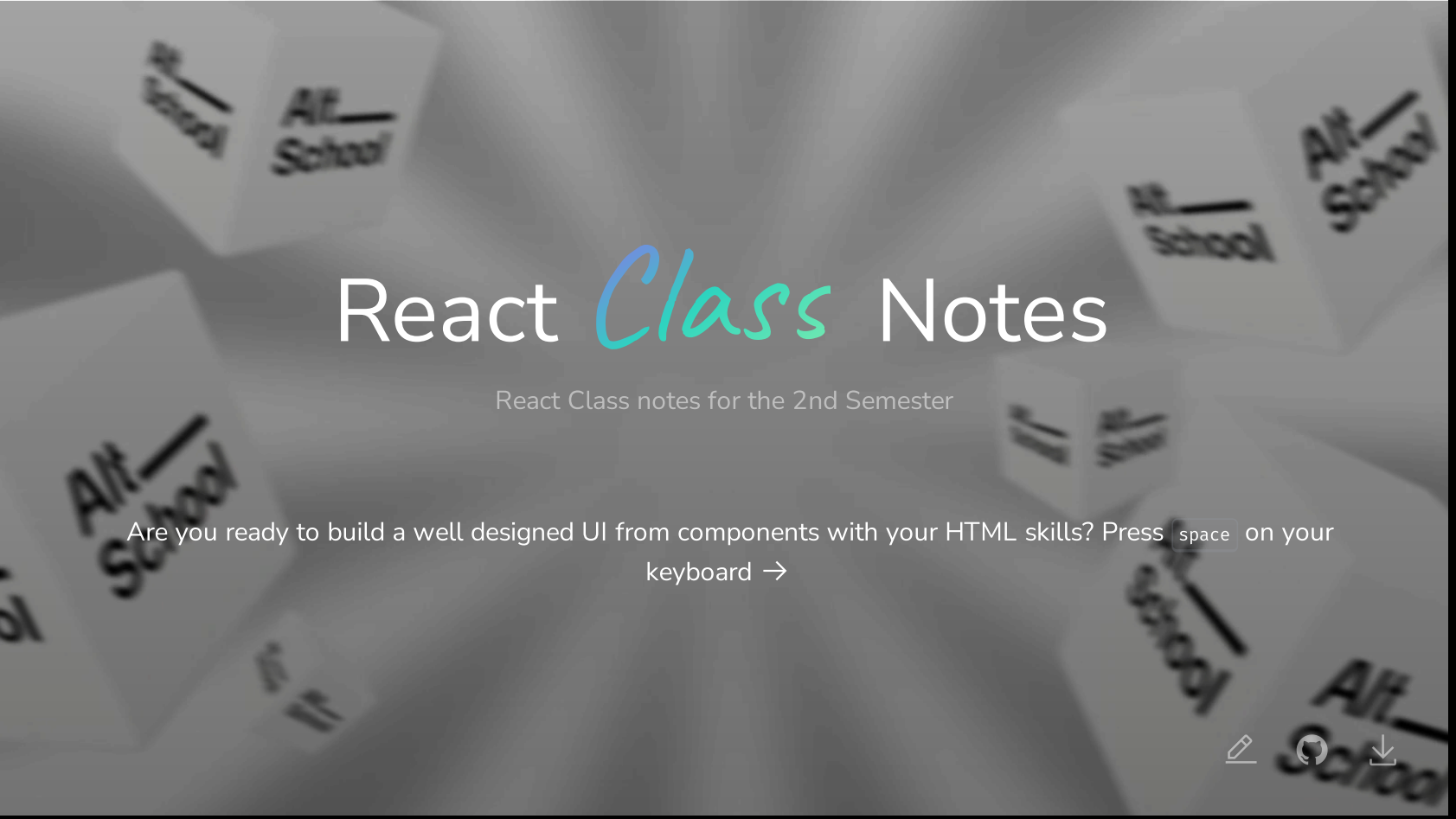


React Class Notes

React Class notes for the 2nd Semester

Are you ready to build a well designed UI from components with your HTML skills? Press `space` on your keyboard →

Table of contents

1. React Class Note
2. Introduction
3. JSX
4. Component
5. Interaction
6. Rendering
7. Hooks
8. State
9. Form
10. More Hooks
11. Routing
12. State Management
13. Data Fetching
14. Performance
15. Test
16. Pagination Order
17. Infinite Scroll Pagination
18. Pagination GraphQL
19. Router Example
20. Assignments

Introduction to Libraries ReactJS, VueJS

The library for web and native user interfaces

Introduction

- What is a Library/Framework?
- What is React?
- React vs other frameworks (Vue)
- Why React?
- From pure javascript to React
- Setting up Development Environment (Vite)

What is a Library/Framework?

Javascript libraries and frameworks are tools that help developers build applications faster and more efficiently. They provide a set of functionalities that can be reused across different projects, saving time and effort.

Library is a collection of pre-written code that developers can call upon to perform specific tasks. Libraries offer various functionalities, but you are in control of how and when to use them.  React is often considered a library because it focuses on rendering the UI. It doesn't provide everything needed to build an application, but it does provide a way to build user interfaces.  Vue,  Angular, and Svelte are other examples of libraries.  jQuery is another example of a library that provides functionalities like DOM manipulation, event handling, and AJAX calls.

Framework A framework is a more opinionated structure or skeleton for building applications. It typically provides everything needed (like a routing system, data management, etc.) and controls the flow of the application. Although  Nextjs uses  React, it is considered a framework because it provides a predefined structure for building applications, including file-based routing, server-side rendering, and static site generation.  Nuxt is another example of a framework that uses  Vue.

What is React?

React is a JavaScript library for building user interfaces. It is maintained by Facebook and a community of individual developers and companies.

React allows developers to create reusable UI components and manage the state of the application. React uses a declarative approach to build components, which means you describe what you want to see, and React takes care of rendering the UI. React is used to build single-page applications, progressive web apps, and mobile applications.

Some of the key features of React include:

- **Components**: React allows developers to create reusable UI components that can be used across different parts of the application.
- **Virtual DOM**: React uses a virtual DOM to improve performance by updating only the parts of the UI that have changed.
- **JSX**: React uses JSX, a syntax extension that allows developers to write HTML-like code in JavaScript.

- **State Management**: React provides a way to manage the state of the application using hooks and context API.
- **Lifecycle Methods**: React provides lifecycle methods that allow developers to hook into different stages of the component lifecycle.
- **Server-side Rendering**: React supports server-side rendering, which improves the performance and SEO of the application.
- **Server Components**: React 19 introduces server components, which allow developers to build components that can be rendered on the server and sent to the client. Uses Server Actions to fetch data on the server and send it to the client with heavy support for streaming and partial hydration.

In summary, React allows developers to create user interfaces from components, write components with code and markup in JSX, add interactivity with events, state and props. Going further to full-stack capabilities with server components and server actions for better data fetching and rendering. Allowing developers to build fast, interactive, and scalable applications.

React vs other frameworks (Vue)

React and Vue are two popular JavaScript libraries for building user interfaces.

Both libraries have their strengths and weaknesses, and the choice between them depends on the requirements of the project and the preferences of the developers. During the course of this workshop, we will be focusing on React in the second semester, later on we will explore Vue but it is worth noting some of the key differences between React and Vue:

- **Learning Curve**: React has a steeper learning curve compared to Vue because it requires developers to learn JSX, hooks, and other concepts. Vue, on the other hand, is easier to learn because it uses a template-based syntax that is similar to HTML even though it has composables and support JSX as well.
- **Performance**: React uses a virtual DOM to improve performance by updating only the parts of the UI that have changed. Vue also uses a virtual DOM but has a more optimized reactivity system that can update the UI more efficiently especially with its vapor rendering.
- **State Management**: React provides a way to manage the state of the application using `useState` hooks and context API. Vue also provides a way to manage the state of the application using Pinia, which is a state management library.

Counter Example in React

```
function Hello() {
  const [counter, setCounter] = React.useState(0)
  const value = 2
  const doubled = counter * value

  return (
    <div className="select-none text-lg flex gap-4 items-center p2 border border-main">
      <span className="text-gray text-lg">
        <span className="text-orange">{counter}</span> * {value} ={' '}
        <span className="text-green">{doubled}</span>
      </span>
      <button
        className="border border-main p2 rounded"
```

0 * 2 = 0 +1 -1

Counter Example in Vue

```
<script setup>
import { computed, ref } from 'vue'
const counter = ref(1)
const value = 2
const doubled = computed(() => counter.value * value)
function inc() {
  counter.value++
}
</script>

<template>
  <div
    class="select-none text-lg flex gap-4 items-center p2 border border-main"
```

1 * 2 = 2 +1 -1

Why React?

React has become one of the most popular JavaScript libraries for building user interfaces. There are several reasons why developers choose React over other libraries and frameworks:

- **Simple and Less opinionated** : React is a lightweight library that focuses on rendering the UI. It doesn't provide everything needed to build an application, but it does provide a way to build user interfaces. This makes it easier to learn and use compared to other libraries and frameworks.
- **Popular** : It has a wide spread adoption and a large community of developers and companies that contribute to its development and maintenance. This means that there are plenty of resources available for learning React and getting help when needed. Jobs are also available for React developers.
- **Performance** : React uses a virtual DOM to improve performance by updating only the parts of the UI that have changed. This makes React faster and more efficient than other libraries and frameworks. If its becomes slow then its on you the developer.
- **Flexibility** : React is flexible and allows developers to build applications the way they want.

From pure javascript to React

```
const makeElem = (elemType, props, children) => {
  const elem = document.createElement(elemType)

  if (props) {
    for (const [key, value] of Object.entries(props)) {
      if (key === 'onclick' && props['once']) {
        elem.addEventListener('click', value, { once: true })
      } else {
        elem[key] = value
      }
    }
  }

  if (children) {
    elem.prepend(...children)
  }

  return elem
}

const createTodo = (todo) => {
  const text = makeElem(
```

```
<span style="flex: 1 1 0%;">Hello World</span><button>X</button>
```

From pure javascript to React

```
// just the ui
// replace the mk() with React.createElement
// const mk = React.createElement()
const App = () => {
  return React.createElement(
    'div',
    { id: 'app' },
    React.createElement(
      'form',
      { onSubmit: createTodo },
      React.createElement('input'),
      React.createElement('button', { type: 'submit' }, 'Add ToDo'),
    ),
    React.createElement('ul', { style: { padding: '5px' } }),
  )
}
```

```
// ui and state
const App = () => {
  const [todos, setTodos] = React.useState([])
  const [id, setId] = React.useState(0)
  const [text, setText] = React.useState('')
  const [editingId, setEditingId] = React.useState(null)
  const [editText, setEditText] = React.useState('')

  return React.createElement(
    'div',
    { id: 'app' },
    React.createElement(
      'form',
      { onSubmit: createTodo },
      React.createElement('input', {
        value: text,
        onChange: (e) => setText(e.target.value),
      }),
      React.createElement('button', { type: 'submit' }, 'Add ToDo'),
    ),
    React.createElement(
      'ul',
      { style: { padding: '5px' } },
      todos.map((todo) =>
        React.createElement(
          'li',
          { style: { display: 'flex' }, key: todo.id },
          editingId === todo.id
            ? React.createElement('input', {
```

```
// ui + state + event handlers
const App = () => {
  const [todos, setTodos] = React.useState([])
  const [id, setId] = React.useState(0)
  const [text, setText] = React.useState('')
  const [editingId, setEditingId] = React.useState(null)
  const [editText, setEditText] = React.useState('')

  const createTodo = (e) => {
    e.preventDefault()
    if (!text.trim()) return
    setTodos([...todos, { id: id, text: text, completed: false }])
    setId(id + 1)
    setText('')
  }

  const remove = (id) => setTodos(todos.filter((todo) => todo.id !== id))

  const startEditing = (todo) => {
    setEditingId(todo.id)
    setEditText(todo.text + '')
  }

  const saveEdit = () => {
    setTodos(
      todos.map((todo) =>
        todo.id === editingId ? { ...todo, text: editText } : todo,
      ),
    )
```

```
// ui + state + event handlers + render
const App = () => {
  const [todos, setTodos] = React.useState([])
  const [id, setId] = React.useState(0)
  const [text, setText] = React.useState('')
  const [editingId, setEditingId] = React.useState(null)
  const [editText, setEditText] = React.useState('')

  const createTodo = (e) => {
    e.preventDefault()
    if (!text.trim()) return
    setTodos([...todos, { id: id, text: text, completed: false }])
    setId(id + 1)
    setText('')
  }

  const remove = (id) => setTodos(todos.filter((todo) => todo.id !== id))

  const startEditing = (todo) => {
    setEditingId(todo.id)
    setEditText(todo.text)
  }

  const saveEdit = () => {
    setTodos(
      todos.map((todo) =>
        todo.id === editingId ? { ...todo, text: editText + ' □' } : todo,
      ),
    )
```

0 @Oluwasetemi

Add ToDo



Click to load iframe

https://stackblitz.com/edit/web-platform-zxxs4v?ctl=1&embed=1&file=script.js&hideExplorer=1&hideNavigation=1



Click to load iframe

https://stackblitz.com/edit/vitejs-vite-epzrqa?ctl=1&embed=1&file=main.js&hideExplorer=1&hideNavigation=1

Setting up Development Environment Vite

Let us talk about build tools, they are tools that help developers build and deploy applications.

They automate the process of compiling, bundling, and optimizing code. Some popular build tools include webpack, turporepo, Parcel, and Vite(Rolldown, Rollup, esbuild,), and Rspack.

Vite is a build tool that is designed to be fast and lightweight. It uses ES modules to improve performance and provides a zero-config setup for building modern web applications. Vite supports React, Vue, and other frameworks and libraries.

To set up a development environment with Vite, you can use the following steps:

1. Install Vite globally(once and for all) or locally (per project) using npm or yarn or pnpm or bun:

```
bun add -g create-vite
bunx create-vite
```

2. Create a new project using Vite:

```
create-vite my-app
cd my-app
```

3. Start the development server and Open your browser and navigate to `http://localhost:5173`.

```
yarn run dev
```

4. Build the project for production and Deploy the project to a hosting service like Netlify, Vercel, or GitHub Pages.

```
yarn run build
```

5. You can also customize the build configuration by creating a `vite.config.js` file in the root of the project.

Setting up everything from scratch

If you prefer to set up everything from scratch, you can use the following steps:

1. Create a new project directory and navigate to it:

```
mkdir my-app
cd my-app
```

2. Initialize a new npm or yarn or pnpm or bun project:

```
bun init -y
```

3. Install React and React DOM:

```
bun i react{@latest,-dom@latest}
```

4. Create an `index.html` file in the root of the project:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" href="https://fav.farm/□□" />
    <title>My App</title>
  </head>

  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

5. Create a `src` directory in the root of the project and create a `main.jsx` file in the `src` directory:

```
import React from 'react'
import { createRoot } from 'react-dom/client'

// function App() {
//   return <h1>Hello World</h1>;
// }

function App() {
  return React.createElement('h1', null, 'Hello World');
}

const root = createRoot(document.getElementById('app'));

root.render(<App />);
```

6. Install Vite as a development dependency and @vitejs/plugin-react

```
npm|pnpm|bun|yarn install -D vite@latest @vitejs/plugin-react@latest
```

7. Create a `vite.config.js` file in the root of the project:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
})
```

8. Setup the build script in the `package.json` file add the following to the file:

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  }
}
```

9. Start the development server and Open your browser and navigate to `http://localhost:5173`.Use pnpm or bun or yarn.

```
npm run dev
```

10. Build the project for production and Deploy the project to a hosting service like Netlify, Vercel, or GitHub Pages.

```
npm run build
```

You can use online editors like Codesandbox, Codepen, or Stackblitz to create and share React applications without setting up a development environment on your local machine.

Javascript XML syntax, structure, rules

HTML ➡ JSX

JSX

- Ways of writing HTML in a JavaScript file
- What is JSX? Syntax and Structure
- The Rules of JSX
- Convert HTML to JSX
- Embedding JavaScript in JSX
- CSS in JSX

Ways of writing HTML in a JavaScript file

Ever wondered how you can write HTML in a JavaScript file?

There are several ways to achieve this which include:

1. Using `Template Literals (``)`: Template literals allow you to embed HTML directly into JavaScript using backticks. They provide an easy way to write multi-line HTML strings.

```
const content = `
  <div class="container">
    <h1>Hello, World!</h1>
    <p>This is a paragraph.</p>
  </div>
`

document.body.innerHTML = content
```

Hello, World!

This is a paragraph.

2. `document.createElement` Method: This method creates HTML elements dynamically using JavaScript.

```
const div = document.createElement('div')
div.className = 'container'

const h1 = document.createElement('h1')
h1.textContent = 'Hello, World!'

const p = document.createElement('p')
p.textContent = 'This is a paragraph.'

div.appendChild(h1)
div.appendChild(p)
document.body.appendChild(div)
```




```
// a function to create html elements
const makeElem = (elemType, props, children) => {
  const elem = document.createElement(elemType)

  if (props) {
    for (const [key, value] of Object.entries(props)) {
      if (key === 'onclick' && props['once']) {
        elem.addEventListener('click', value, { once: true })
      } else {
        elem[key] = value
      }
    }
  }

  if (children) {
    elem.prepend( ...children)
  }

  return elem
}

window.makeElem = makeElem
```

3. Using the `innerHTML` Property: You can directly set the `innerHTML` property of a DOM element with a string containing HTML.

```
const container = document.getElementById('app')
container.innerHTML = `
  <h1>Hello, World!</h1>
  <p>This is a paragraph.</p>
`
```

4. String Concatenation: You can use traditional string concatenation to build HTML strings.

```
const title = 'Hello, World!'
const paragraph = 'This is a paragraph.'
const content =
  '<div class="container">' +
  `<h1>${title}</h1>` +
  `<p>${paragraph}</p>` +
  '</div>'
document.body.innerHTML = content
```

5. JSX (JavaScript XML) in React

What is JSX?

JSX is a syntax extension for JavaScript used in React. It allows you to write HTML-like syntax directly within JavaScript. Most React developers prefer the conciseness of JSX, and most codebases use it.

Note:

- JSX is not the only way to write HTML in React as seen in previous pages, but JSX makes it easier to write React applications
- JSX converts HTML tags into react elements.
- JSX is an extension used in React, this means that it is not part of react. It is a separate syntactic tool to help React developers create React elements easily
- JSX is a type of expression in JS, not a statement.
- JSX allows you to write HTML elements in JavaScript and place them in the DOM without any createElement() and/or appendChild() methods.

Syntax and Structure of JSX

The syntax of JSX elements looks very much alike with that of HTML elements.

For example, if you want to create a `h1` and a `div` using JSX, they will look the same as with normal HTML:

```
<h1>Hello AltSchoolers , This is a JSX heading and I love it!</h1>

<div>Alright, important question. How about centering a div in JSX? </div>
```

```
// With JSX
const myElement = <h1>You know, I Love JSX!</h1>

const root = ReactDOM.createRoot(document.getElementById('root'))
root.render(myElement)
```

```
// Without JSX
const myElement = React.createElement('h1', {}, 'He said he does not use JSX!')

const root = ReactDOM.createRoot(document.getElementById('root'))
root.render(myElement)
```

The Rules of JSX

There are few rules that guides writing JSX. They include:

1. Return a single root element: This is simply saying you should always wrap a group of JSX elements with a single root.

```
// the `<div></div>` is the parent (root element)
<div>
  <h1>I am the first Element in the group!</h1>
  <p>Well I am the second, but I have details more than you do!</p>
  <img src="imageUrl" alt="I am the bigger picture guys.">
</div>
```

If you don't want to add an extra `<div>` to your markup, you can use `<>` and `</>` instead:

```
// the `<></>` is the parent (root element) called Fragment
<>
  <h1>Our parent Element don't even have a public name</h1>
  <p>Well, everyone knows the name is <b>Fragment</b>!</p>
  <img src="imageUrl" alt="<p> my paddy is always right">
</>
```

2. Close all the tags: In JSX, every tag should be explicitly closed. This means that all tags, including self-closing tags like `<img>`, `<link>`, and `<meta>`, should be explicitly closed with the forward-slash (/): `<img />`, `<link />`, and `<meta />`. JSX follows XML rules, and therefore HTML elements must be properly closed. JSX will throw an error if the HTML is not properly closed.

```
// Closing tags properly
<>
  <h1>Making sure all tags are explicitly closed</h1>
  <p>I have got some good news!</p>
  <img src="imageUrl" alt="Thank you for closing me properly in JSX" />
  <input type="text" />
</>
```

3. Use camelCase for most of the things!: JSX converts to JavaScript, and attributes written in JSX become keys of JavaScript objects. As we know, JavaScript has limitations on variable names, hence using camelCase is recommended. For example, their names can't contain dashes or be reserved words like `class`. Since `class` is a reserved word, in React you write `className` instead.

```
// using camelCase in JSX
<>
  <img
    className="mr-image"
    src="imageUrl"
    alt="My class in jsx is written as className"
  />
  <input type="text" className="input-lady" />
</>
```

Convert HTML to JSX

Recall that we stated that `JSX` syntaxes are very similar to that of `HTML`, but being 'very similar' means there are distinctive differences, for example the `class` attribute in HTML is written as `className` in JSX. For this reason, there is need to convert HTML to JSX when migrating from a pure HTML file.

Here are some HTML to JSX conversion tools:

- transform.tools/html-to-jsx
- convertsimple.com/convert-html-to-jsx
- Chrome extention: html-to-jsx



Click to load iframe

https://transform.tools/html-to-jsx

Embedding JavaScript in JSX

There are times you will want to add a little JavaScript logic or reference a dynamic property inside a JSX markup. To achieve this, you simply have to add curly braces (`{}`) which will open up a javascript land/window and allow you to embed a JavaScript logic or reference a dynamic content.

```
function EmbedJavascript() {
  const myHeadingContent =
    'I expect this text content to be embedded to a JSX h1 element!'
  const avatar = '/altschool-logo.png'
  const description = 'AltSchool Africa logo'

  return (
    <div className="select-none flex flex-col items-center p2 border border-main">
      <h1 className="text-gradient !text-xl text-center">{myHeadingContent}</h1>
      <div className="flex gap-4 items-center">
```



CSS in JSX

Just like HTML, you can style JSX with inline-styles, external-style, .module.css, and/or any CSS framework/library.

1. Inline CSS: Inline styles in JSX are written as objects with camelCase properties instead of traditional CSS syntax. See inline styles as adding an object to the 'javascript land' of `className` attribute. This means that we would need double curly braces (`{{}}`); one for embedding javascript, the second for applying an object containing styles.

```
function App() {
  const divStyle = {
    backgroundColor: 'blue', // Use camelCase
    color: 'white',
    borderRadius: '5px',
    padding: '10px',
  }

  return (
    <div style={divStyle}>
```




```
function App() {
  return (
    <div
      style={{
        backgroundColor: 'blue',
        color: 'white',
        borderRadius: '5px',
      }}
    >
      <h1>Hello, World!</h1>
      <p>This is a styled paragraph.</p>
    </div>
  )
```

2. CSS Classes: To apply styles from an external style sheet or a .module.css, you simply need to reference the JSX element you want to style by it's `className`, `id`, or other attributes

```
import ./style.css;

function StyleJSX() {
  return (
    <h2 className="text-blue">Hello World!</h2>
  );
}
```

```
import ./style.module.css;

function StyleJSX() {
  return (
    <h2 className={style.title}>Hello World!</h2>
  );
}
```

3. Using CSS frameworks like TailwindCSS or UnoCSS: In this method, you simply write your styles in the `className` quote according to guidelines.

```
function App() {
  return (
    <div className="bg-blue-500 text-white p-4 rounded">
      <p className="text-base">This is a paragraph with Tailwind CSS.</p>
    </div>
  )
}
```

4. Using styled component: Styled components can also be used to style JSX. You simply have to import the component either from a library or your own styled components folder.

Assignment on JSX

Using HTML to JSX Converter, transform a HTML file you already have to JSX.

Components in React parent, child, props

Reusability and Component Hierarchy

Components in React

- What are components?
- Your first component
- Single root element
- Exporting, Importing, and Nesting Components
- Parent and Child component
- Props: Passing and Reading Data Between Components

What are components

Think of a house.

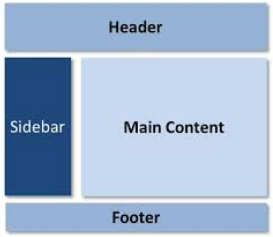


A house is made up of reusable elements: doors, windows, rooms, furniture, lights, and more. You can change one element (like the lights) without affecting others. These are the components of a house. Similarly, React components allow you to divide your UI into logical, reusable parts.

To build the house, you break it down into smaller, manageable parts, just as React breaks a user interface into components. Each part has a specific role, and all these parts work together to form a functional, maintainable whole.

COMPONENTS ARE THE FOUNDATION UPON WHICH USER INTERFACES (UI) ARE BUILT.

In React, a website can be broken into smaller components like a header, sidebar, footer, main content, and buttons. Each component combines its own markup, CSS, and JavaScript, making it unique, reusable, and easy to manage. Together, these components form the building blocks of a complete application.



Your First 1 Component

EVERY REACT COMPONENT IS A FUNCTION.

Let's create a simple functional component that displays a welcome message.

```
// export the component
export default function Greeting() {
  return <p>You are welcome here.</p>
}
```

1. Define the component function: React components are regular JavaScript functions, but their names must start with a capital letter or they won't work!

2. Write return with parenthensis: Always wrap your JSX markup in parentheses after the return statement. Without parentheses, any code on the lines after return will be ignored!

3. Add JSX markup: Markup is your code structure. Inside the component, write your JSX, which looks like HTML but works under the hood as JavaScript.

4. Export the component: The export default prefix is a standard JavaScript syntax (not specific to React). It lets you mark the main function in a file so that you can later import it from other files.

Did you notice that the greeting returned on the previous page was not wrapped in parentheses?

Return statements can be written all on one line, as in this component:

```
function Greeting() {
  return <p>You are welcome here.</p>
}
```

But if your markup isn't all on the same line as the return keyword, you must wrap it in a pair of parentheses:

```
function Greeting() {
  return (
    <div>
      <p>You are welcome here.</p>
    </div>
  )
}
```

Single Root Element

In React, a component can only return 1 root element.

You cannot return multiple elements (like two divs tags) at the root level. However, you can nest elements within a single root element, such as wrapping them in a parent element or using React Fragments . For example:

This is valid ✔

```
function Greeting() {
  return (
    <div>
      <p>Hello, Altschooler.</p>
      <p>You are welcome here.</p>
    </div>
  )
}
```

This is not valid

```
function Greeting() {
  return (
    <p>Hello, Altschoolers.</p>
    <p>You are welcome here.</p>
    <input placeholder="Your name" />
    <button>Submit</button>
  );
}
```

This is valid ✔ (using Fragment)

```
function Greeting() {
  return (
    <>
      <p>Hello, Altschooler.</p>
      <p>You are welcome here.</p>
      <button>Click Me</button>
    </>
  )
}
```

This is how you create a component, no matter how big or small it is. Now that you can create components, how do you use them? ➤

Exporting, Importing, and Nesting Components

Components are powerful because they are reusable. The components created are exported so they can be used (imported) into other files.

Below, the `Greeting` component created is imported into the root App component in React.

```
export default function Greeting() {
  return (
    <div>
      <p>Hello, Altschoolers.</p>
      <p>You are welcome here.</p>
    </div>
  )
}
```

```
import Greeting from './Greeting'

function App() {
  return (
    <div>
      <Greeting />
    </div>
  )
}
```

```
export function Greeting() {
  return (
    <div>
      <p>Hello, Altschoolers.</p>
      <p>You are welcome here.</p>
    </div>
  )
}
```

```
import { Greeting } from './Greeting'
function App() {
  return (
    <div>
      <Greeting />
    </div>
  )
}
```

```
import { Greeting as RenamedGreeting } from './Greeting'
function App() {
  return (
    <div>
      <RenamedGreeting />
    </div>
  )
}
```

Guide for importing components

1. Use the Correct Path: Ensure you provide the correct relative path to the component file you want to import. For example, import Greeting from './Greeting'; if Greeting.jsx is in the same directory or '../Greeting'; if the Greeting.jsx is one level deeper.

2. Case Sensitivity: Component names are case-sensitive. The imported name must match the exported name exactly. For example, import Greeting from './greeting'; will fail.

3. Export Types: For default exports, import without curly braces. For named exports, import with curly braces.

```
// importing default exports
import Greeting from './Greeting'
```

```
// importing named exports
import { Greeting } from './Greeting'
```

4. File Extensions: Use the appropriate file extension. You can omit the .js or .jsx extension when importing, but including it can sometimes improve clarity.

Nesting Components

A nested component involves placing one component inside another to efficiently structure your UI. In the example below, we use three components: **Header**, **Content**, and **Footer**. These components are nested within the `App` component to create a well-organized layout:

```jsx
// File: App.jsx
import Header from './Header'
import Content from './Content'
import Footer from './Footer'

function App() {
  return (
    <div>
      <Header />
      <Content />
      <Footer />
    </div>
  )
}

export default App
```

This enhances reusability across different parts of the app or projects, improves maintainability with an organized layout, and promotes cleaner code by breaking the UI into smaller, manageable pieces.

Parent & Child Component

In React, components often have a parent-child relationship, where the **parent** component contains other components as its **children**. This relationship allows for **data** to flow between components, enabling a structured, reusable, and maintainable UI.

Think of that House again

A room (parent) may contain **furniture** (children) like a bed, chair, or lamp. Each furniture item (child) serves its specific role, but the room (parent) coordinates their placement and usage.

Similarly, in React, a parent component collaborates with its children to build a cohesive application. The previously discussed nested components are the children of the App (parent) component.

A parent component renders one or more child components within it. It can pass data and functions to its children.

A child component is nested within a parent component. It can receive data from the parent to render its UI or perform actions.

Props : Passing and Reading Data Between Components

Props (short for properties) allow parent components to pass data to their child components, just like arguments in functions and attributes in html. You can pass any JavaScript value as a prop (objects, arrays, functions, etc.). Below, child component `Welcome` is receiving name props from parent component `App`

```
function App() {
  return (
    <div>
      <Welcome name="Alice" />
      <Welcome name="Bob" />
    </div>
  )
}

function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>
}

// Output:
// Hello, Alice! // Hello, Bob!
```

How to Use `Props`

1 Pass Props to Components: Add them like HTML attributes.

```
function Profile() {
  return (
    <div>
      <Avatar name="Ojo Setemi" id="1bX5QH6" />
    </div>
  )
}
```

2 Destructure Props: Extract props in the function parameters. Optionally assign default values.

```
function Avatar({ name, id, size = 100 }) {
  return (
    <img
      src={`https://i.imgur.com/${id}.jpg`}
      alt={name}
      width={size}
      height={size}
    />
  )
}
```

Forwarding Props with Spread Syntax: You can forward all props to child components with ...props

```
function Profile() {
  return (
    <div>
      <Avatar {...props} />
    </div>
  )
}
```

```
function Profile({ name = 'default value', ...props }) {
  return (
    <div>
      <Avatar {...props} />
    </div>
  )
}
```

Special Props `children`

The `children` prop is a special prop that allows you to pass components as data to other components. It is used to render whatever you include between the opening and closing tags of the component.

```
function Card({ children }) {
  return <div className="card">{children}</div>
}

function App() {
  return (
    <Card>
      <h1>Card Title</h1>
      <p>Card Content</p>
    </Card>
  )
}
```

```
function Card({ title, children }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      {children}
    </div>
  )
}

function App() {
  return (
    <Card title="Card Title">
      <p>Card Content</p>
    </Card>
  )
}
```

Assignment

Build a simple information display application that consists of a Header component with a title (e.g., "My Favorite Books"), a BookList component to list three of your favorite book titles, and a Book component for each individual book. Each Book component should accept a title prop and display the title as text.

Turn this `li` into a reusability component.

```
<li className="contact-card">
  <h2>Sunita Kumar</h2>
  <dl>
    <dt>Job</dt>
    <dd>Electrical Engineer</dd>
    <dt>Email</dt>
    <dd>sunita.kumar@acme.co</dd>
  </dl>
</li>
```

Implement either an accessible modal, accordion and tool tip component

Summary

- Try to read the codebase of React Component Libraries from React Aria, Chakra UI, Material UI.
- Check out Tanstack family of libraries.

Make a Component Interactive event handlers

Passing event handlers as props

Interactivity

- How do you make a component interactive? - events
- Creating and adding event handlers
- Dos and Don’ts
- Passing event handlers as props
- Naming event handler props
- Propagation

How to make a component interactive? - events

In the previous chapter we learned about Components in React and how to create one. Let's talk about making them interactive.



Bringing back our components mansion (the house), we mentioned that the components of a house are reusable features like the doors, windows, lights, etc. These features needs human interaction to be usable or to act in a certain way. For example, if you want to enter or leave the house, you need to open the door (interact with the door), you can turn on/off the light as you desire (interactivity).

The same logic of interactivity about a house above applies to React components. For example, you may desire to alert a success message to your users only when they click a button. This can be achieved when you add an event to the button to listen to the user's interaction. When the data of a JSX element or a component change over time automatically or due to user interaction, we call it "changing of state".

Now that we mentioned Events, let's talk about Events in React.

Making a Component Interactive with Events

You can add events directly to JSX elements rendered by your component. In the example below, the `onClick` event is used to listen for a click interaction and apply a `state` update to the component using `setCount`. The `useState` hook is used to update the state of a component in React. We will talk more on `useState` when learning about Hooks and State Management in React.

```
function Counter() {
  const [count, setCount] = React.useState(0)

  return (
    <div className="flex gap-3 items-center">
      <button onClick={() => setCount(count + 1)} className="btn">
        Click Me
      </button>
      <p>
        You clicked <span className="text-gradient">{count}</span> times
      </p>
    </div>
```

Click Me You clicked 0 times

Roles of State in a React Component's Interactivity

State is regarded as the memory of a React component. When a component gains interaction, it compares its previous form before the interaction, and its form after the interaction. This action can be termed as *"Comparing of State Change in the component"*. When React compares the before and after form of a component during *State Change* and they differ, React goes ahead to update the component with the latest changes.

You can add a State to your component using the `useState` Hook React provides. Hooks are helper functions that lets you use some special features in React. State updates based on Interactivity.

Roles of State

- Triggering UI Updates
- Tracking User Input
- Handling Component Visibility
- Enabling Conditional Rendering
- Managing Component Animation and Transitions

```
function FadeComponent() {
  const [visible, setVisible] = React.useState(false)

  return (
    <div className="flex gap-3 items-center">
      <button className="btn" onClick={() => setVisible(!visi
        Toggle Visibility
      </button>
      <div className={visible ? 'block text-gradient' : 'fade
        Fading Text
```

Toggle Visibility

Creating and Adding Event Handlers to JSX in React

React allows you add *event handlers* to JSX. Event handlers are functions we created to be triggered when a user interacts with a `Component` or a `JSX` element like clicking, hovering, focusing on a `button`.

```
function handleClick() {
  console.log('Button clicked');
}

<button onClick={handleClick}>Click me</button>
```

In the above codeblock, we created our *event handler* (`handleClick()`) and passed it to the `onClick` prop of the `button` element to console log 'Button clicked' when a user clicks the button.

Dos and Don’ts

Dos:

- Use Arrow Functions for Simplicity: Especially when you need to pass arguments to the event handler.

```
<button onClick={() => handleClick(item.id)}>Click me</button>
```

- Use `event.preventDefault()`: When you want to prevent the default behavior of events like form submissions or link clicks.

```
function handleSubmit(event) {
  event.preventDefault()
  // Handle form submission logic
}
```

- Keep Event Handlers Small: If your event handler logic grows too large, consider moving it into a separate function for better readability.

Dos and Don’ts

Don’ts:

- Do Not Use Inline Handlers for Complex Logic: Avoid putting long or complex logic directly inside JSX as it can make the code harder to read.

```
// Do
function Product({ price, discount }) {
  const calculateDiscount = () => {
    const discountedPrice = price - (price * discount) / 100
    if (discountedPrice < 0) {
      console.log('Discounted price cannot be negative')
    } else {
      console.log(`Discounted price is $${discountedPrice}`)
    }
  }
  return (
    <div>
      <p>Price: ${price}</p>
      <p>Discount: {discount}%</p>
      <button onClick={calculateDiscount}>Calculate Discount</button>
    </div>
  )
}
```

Dos and Don’ts

Don’ts:

- Do Not Overuse Arrow Functions in the JSX: Inline arrow functions can cause issues in performance-sensitive parts of the application because a new function is created on every render. This can lead to unnecessary re-renders of child components.

```
// Do
function UserList({ users, onSelectUser }) {
  const handleSelectUser = (id) => {
    onSelectUser(id)
  } // Define the function outside of the map and pass it to the child elements

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>
          {user.name}
          <button onClick={handleSelectUser(user.id)}>Select</button>
        </li>
      ))}
    </ul>
  )
}
```

Passing Event Handlers as Props

Built-in components like `<button>` only support built-in browser events like `onClick`. So in order to pass an *event handler* as a prop with a desired name to a component, create a custom component and give their *event handlers* props specific names.

```
// Custom component
function ToggleSwitch({ onToggle }) {
  return (
    <button onClick={onToggle}>
      Toggle
    </button>
  );
}
```

```
function App() {
  const handleToggle = () => {
    console.log('Toggle button clicked!')
  }

  return <ToggleSwitch onToggle={handleToggle} />
}
```

- `ToggleSwitch` is a custom component.
- The prop `onToggle` is a custom *event handler* prop that is not tied to a specific browser event. Instead, it is a named function prop that triggers when the button is clicked.
- `handleToggle` is passed as the `onToggle` prop to `ToggleSwitch`.
- When the button inside `ToggleSwitch` is clicked, it triggers `handleToggle`, logging "Toggle button clicked!" to the console.

Naming Event Handler Props

When passing *event handlers* as props, it's best to follow a consistent naming convention:

- Use names that make it clear what action will be triggered, like `onButtonClick`, `onSubmit`, or `onToggle`.
- For custom components, use `on` followed by a descriptive event name passed as props:

```
<MyCustomComponent onAction={() => console.log('Action triggered')} />
```

This follows the same convention as built-in DOM events like `onClick` and `onChange`.

Propagation

Event propagation refers to how events travel through the DOM tree, involving two phases: capturing and bubbling.

- Event Bubbling: An event starts from the target element and propagates upward to its parent elements.

  This is the default in React.

```
function Parent() {
  const handleParentClick = () => {
    console.log('Parent clicked')
  }

  return (
    <div onClick={handleParentClick}>
      <Child />
    </div>
  )
}

function Child() {
  const handleChildClick = (event) => {
    event.stopPropagation() // Prevents the event from reaching the parent
    console.log('Child clicked')
  }

  return <button onClick={handleChildClick}>Click Child</button>
```

- Event Capturing: You can use capture to handle the event during the capture phase before it reaches the target.

```
<div onClickCapture={handleClick}>
  // This will capture the event before it reaches its target.
</div>
```

- Stopping Propagation: Use `event.stopPropagation()` if you want to prevent the event from bubbling up to parent elements. However, use this sparingly as it can cause unexpected behavior in some cases.

React Lifecycle mount, trigger, render, commit

Conditional rendering

Rendering

- Lifecycle (Mount, Trigger, Render and Commit)
- Batching
- Rendering lists
- Conditional rendering - if-else, ternary, && operator
- Conditionally assigning values to a variable

React Lifecycle (Mount, Trigger, Render and Commit)

Components built in React passes through some stages before being displayed on the browser. These stages of rendering components are called *React Lifecycle*. Declarative approach to rendering is what React follows. Developers describes what the UI should look like in their code while React takes care of how the components are rendered on the screen.

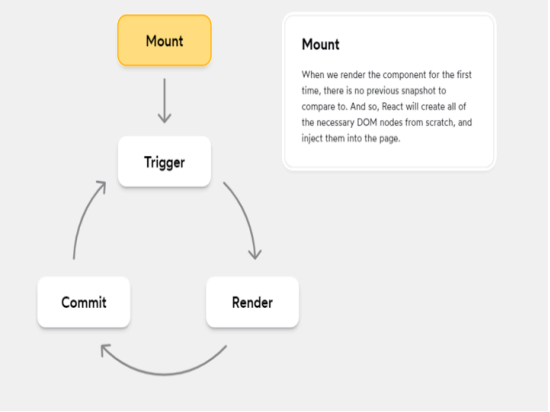
React Components Rendering Stages:

1. Mount:

This is the initial placement of components in Reacts Virtual DOM at the `createRoot` level. By this, we mean that React takes the components and creates a *snapshot* to populate actual Browser DOM nodes at the *FIRST* rendering of the components.

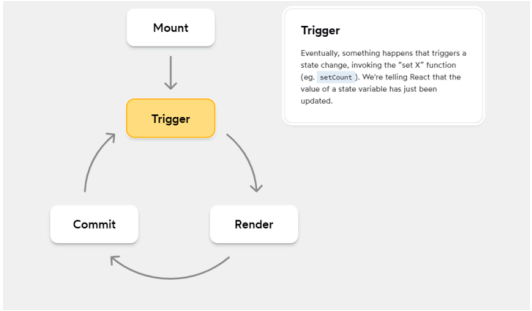
```
import Message from './components/Message.js'
import { createRoot } from 'react-dom/client'

const root = createRoot(document.getElementById('root'))
root.render(<Message />)
```



2. Trigger:

At this stage, React watches for *interactivity* within mounted components so as to *trigger* updates. Undoubtedly, the only thing that can trigger a re-render in a React application is when there is a *State Change*, and this change in state are caused by either automated or user *interactivity* like clicking a button or setting a time interval for updates.




```
function Clock() {
  const [time, setTime] = React.useState(getCurrentTime())

  React.useEffect(() => {
    const interval = setInterval(() => {
      setTime(getCurrentTime())
    }, 1000)

    return () => clearInterval(interval)
```

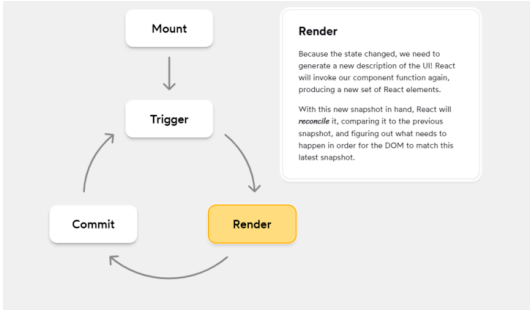
6:22:06 PM

In this example code, Only the time is being updated every second, not the entire application. This is why you can type on the input and the values don't refresh as the time is updated.

During a trigger, React checks to see what part of the component changed, and thus, re-renders only that part. In this case, time state is being updated with `setTime` within an interval.

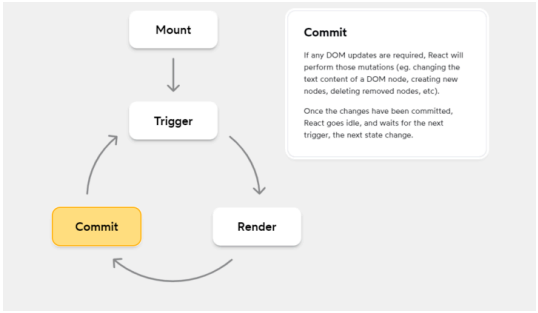
3. Render:

This is the stage where React calls the components to know what to display on the screen. At first trigger (mount), React renders from the `root` which is the entire components. But during a *state change* trigger, React searches for the part that changed and updates only that part as seen in the example code from previous slide.



4. Commit:

React after rendering updates, *commits* them to the changes to actual DOM for us to see. During first trigger (mount), *committing* changes is the same as appending the components to the DOM using the `appendChild()` API. While subsequent triggers due to *interactivity* are committed to the UI based on the calculations of the part that actually changed and needs to be updated.

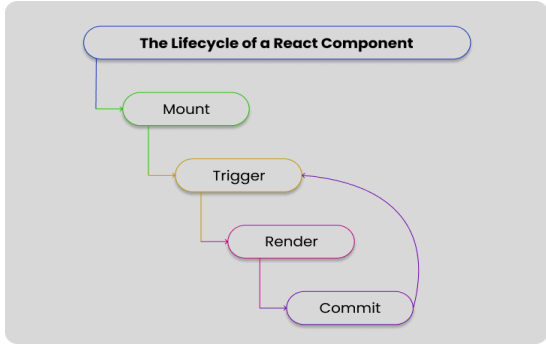


Lifecycle Summery

Step 0: Mount with `ReactDOM`

Step 1: Trigger a render

- It's the component's initial render.
- The component's (or one of its ancestors') state has been updated.



Step 2: React renders your components

Step 3: React commits changes to the DOM

Any screen update in a React app happens in three steps: Trigger, Render, and Commit. You can use `Strict Mode` to find mistakes in your components caused by impure functions because React calls each component twice in `Strict Mode`. React does not touch the DOM if the rendering result is the same as last time.

Batching

In React, batching refers to the process of grouping multiple state updates into a single re-render to optimize performance. Whenever multiple state changes occur within the same event, React automatically *batches* them to reduce the number of renders, which enhances efficiency.

How Batching Works in React

1. Event Handlers: React *batches* state updates within event handlers. If you update the state multiple times within a single event handler, React will group these updates and trigger a single re-render.

```
function Counter() {
  const [count, setCounter] = React.useState(0)

  return (
    <>
      <h1 className="!text-[1.2rem] !mb-0">{count}</h1>
      <button
        className="btn"
        onClick={() => {
```

0

+3

```
function Counter() {
  const [count, setCounter] = React.useState(0)

  return (
    <>
      <h1 className="!text-[1.2rem] !mb-0">{count}</h1>
      <button
        className="btn"
        onClick={() => {
```

0

+3

2. Promises, Async Calls, and Timeouts: Before React 18, batching did not automatically happen in async contexts (e.g., setTimeout, Promise.then). In React 18 and later, however, automatic batching extends to async functions and other async situations, which minimizes unnecessary renders across all scenarios.

```
function RequestTracker() {
  const [pending, setPending] = React.useState(0)
  const [completed, setCompleted] = React.useState(0)

  // batching extended to async event handler wrongly
  async function handleClick() {
    setPending(pending + 1)
    await delay(3000)
    setPending(pending - 1)
    setCompleted(completed + 1)
  }

  return (
    <>
```

Pending: 0
Completed: 0
Buy

```
function RequestTracker() {
  const [pending, setPending] = React.useState(0)
  const [completed, setCompleted] = React.useState(0)

  // batching extended to async event handler correctly
  async function handleClick() {
    setPending((pending) => pending + 1)
    await delay(3000)
    setPending((pending) => pending - 1)
    setCompleted((completed) => completed + 1)
  }

  return (
    <>
```

Pending: 0
Completed: 0
Buy

Bypass batching with `flushSync`

There are times you might want to bypass batching and apply an update immediately. React provides the `flushSync` function to do this.

```
import { flushSync } from 'react-dom'

function ImmediateUpdateExample() {
  const [count, setCount] = useState(0)

  const handleClick = () => {
    flushSync(() => setCount(count + 1)) // This update will be applied immediately
    flushSync(() => setCount(count + 2)) // Another immediate update
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Increment Immediately</button>
    </div>
  )
}
```

Use flushSync sparingly because it forces React to re-render immediately, bypassing the benefits of batching and impacting performance. It's useful in specific scenarios, such as when you need an immediate update for animations or measuring the DOM.

Rendering lists

Often times we would want to render a list of items from an array or a database. This is usually accomplished with the `li` element in `HTML`. Rendering a list from an array of items in React requires a careful procedure:

- You can render a list of items from an array using `map()` to loop over the items and create a `li` of each item in the DOM, and you can filter specific data with `filter()` before mapping.
- Each `li` item should always have a `key` which serves as a unique identifier.

```
const people = [
  { name: "Alice Johnson", job:"Software Engineer" },
  { name: "Brian Thompson", job: "Graphic Designer" },
  { name: "Carlos Rivera" job: "Data Scientist" },
  { name: "Dana Lee" job: "Marketing Specialist" },
];

function List () {
  const filteredPeopleArray = people.filter(person => person.job =/= "Marketing Specialist")
  return (
    <>
      <h1>List of people and their occupation</h1>
      <ul>{filteredPeopleArray.map((person) => <li key={person.name}>{person.name}: {person.job}</li>)}</ul>
    </>
  )
```

Why List key? Where to find a `key` value?

A list `key` is a unique identifier that gives the list item a name in the DOM so as to precisely locate it when needed. Some suggestions of getting a unique `key` value include:

- Using the Serial No. or ID of the item as it is on the database
- Using a unique key generator like `crypto.randomUUID()` or a package like uuid. Although generating key values locally can be tricky since they may change on every re-render, they are best used for local items.

Rules of keys

- Keys must be unique among siblings. However, it's okay to use the same keys for `JSX` nodes in different arrays.
- Keys must not change or that defeats their purpose! Don't generate them while rendering.

Conditional rendering - if-else, ternary, && operator

Sometime we want to trigger a state change based on a certain condition in our application. When the stated condition is met, a specified part of our application gets rendered or re-rendered. This act is term *Conditional Rendering*.

In React, we can render our `JSX` based on a condition in various ways which include:

- Using `if-else` statement

```
function Greeting({ isLoggedIn }) {
  if (isLoggedIn) {
    return null // condition returning nothing
  }
  return <h1>Please sign in.</h1>
}
```

- Using Ternary Operator (`condition ? do : else-do`)

```
function Greeting({ isLoggedIn }) {
  return (
    <h1>{isLoggedIn ? 'Hello!' + □ : 'Hello!'}</h1>
  );
}
```

- Using `&&` (AND) Operator
- (`Left && Right must match`)

```
function Notification({ hasMessages }) {
  return (
    <div>
      <h1>Dashboard</h1>
      {hasMessages && <p>You have new messages!</p>}
    </div>
  )
}
```

Conditionally assigning values to a variable

We can assign `JSX` to a variable based on a condition. As we know in JavaScript, we can re-assign values to variables declared with the `let` keyword. In that regard, we can declare what we want to assign based on a condition, then use any conditional statement to re-assign value or `JSX` that would be rendered to the UI. We can also assign `JSX` directly for cleaner code when using conditional statement like the Ternary Operator.

```
// Example with &&
function Notification({ hasMessages }) {
  const message = hasMessages && <p>You have new messages!</p>

  return (
    <div>
      <h1>Dashboard</h1>
      {message}
    </div>
  )
}
```

Resources

React Lifecycle:

- https://react.dev/learn/render-and-commit
- https://ui.dev/why-react-renders
- https://kentcdodds.com/blog/fix-the-slow-render-before-you-fix-the-re-render

Introduction to React Hooks useState, useEffect

Mutation and Immutability

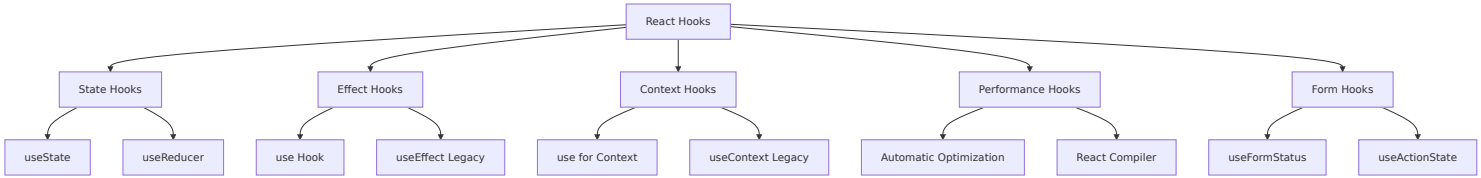
React Hooks

- A New Chapter For React Hooks
- What is state?
- Introduction to React Hooks
- Basic Hooks: "useState", "useEffect"
- Managing Side Effects with "useEffect"
- Complex state (Right way to update objects and arrays in state) - Mutation
- Immutability
- Rules of Hooks

A New Chapter in Simplicity

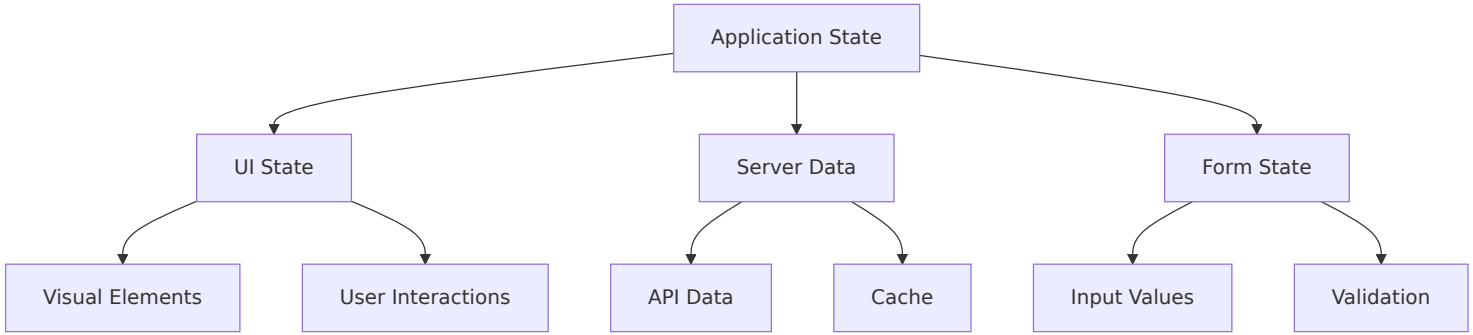
- React Hooks let you add state, side effects, and other React features to your components without writing classes.
- They provide a simpler mental model for sharing logic, reusing behavior, and organizing component code by concern instead of lifecycle methods.
- Hooks replace many patterns from class components, making modern React code more predictable, testable, and easier to refactor.
- In the following slides, you'll see how hooks model state, effects, and complex behavior in a clean, composable way. ⚡
- They heavy build on a concept in JavaScript called closures and here is a video showing how to implement hooks in a very simple way.

Mental Model Overview



What is state

State is data that can change over time in your application. Think of it as the "memory" of your component.



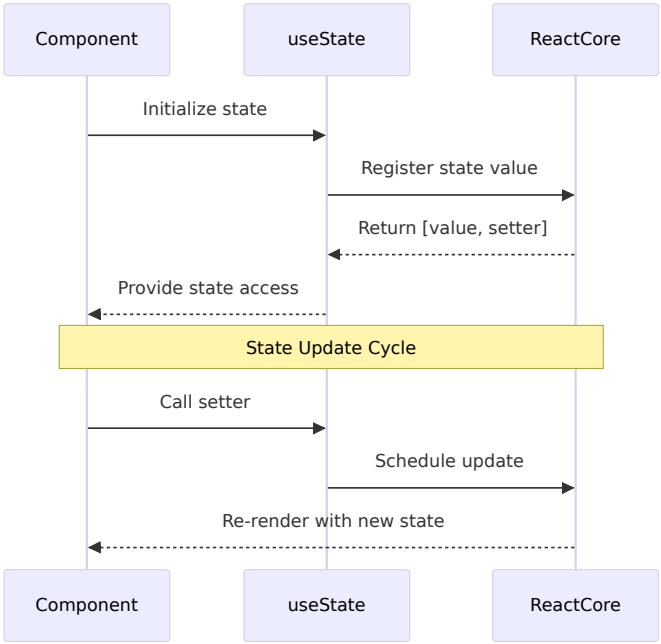
Key Characteristics of State:

- Mutable: Can change over time
- Persistent: Survives re-renders
- Reactive: Changes trigger re-renders
- Isolated: Local to component (unless shared)

Understanding useState

`useState` is a React Hook that lets you add a state variable to your component.

```
const [state, setState] = useState(initialState)
```



React useState Concepts

CONCEPT	EXPLANATION	EXAMPLE	COMMON PITFALLS
Basic Usage	Creates a state variable and its setter function	`const [count, setCount] = useState(0);`	Forgetting to use the s function
Direct Updates	Set the state to a specific new value	`setCount(5);`	Modifying state directly without setter
Functional Updates	Update state based on previous value	`setCount(prev ⇒ prev + 1);`	Not using functional up when depending on pr state
Placement Rules	Must be at top level of component	`function Component() { const [state, setState] = useState(0); }`	Putting useState in conditions or loops

React State Management Explained

```jsx
function Counter() {
  // 1. State Declaration
  const [count, setCount] = useState(0);
  // 2. Update Functions
  const increment = () => setCount((prev) => prev + 1);
  const decrement = () => setCount((prev) => prev - 1);
  // 3. UI Rendering
  return (
    <div className="counter">
      <button onClick={decrement}>-</button>
      <span>{count}</span>
      <button onClick={increment}>+</button>
    </div>
  );
}
```

1. State Declaration

- `useState(0)` creates a new state variable
- Returns array with: [currentValue, updateFunction]
- Initial value is 0

2. Update Functions

- Use function form `(prev => prev + 1)` for reliable updates

Practice Question

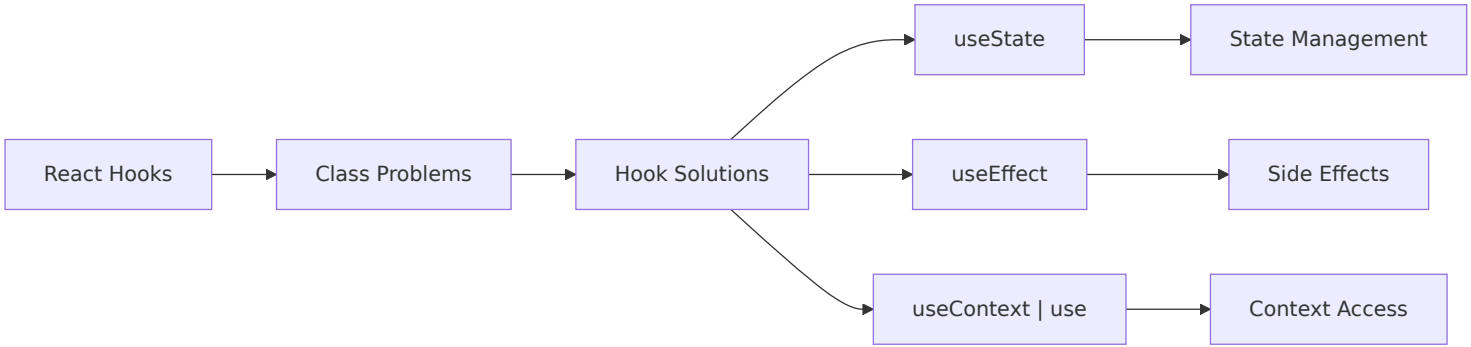
Create a Toggle component that toggles a text. Check this page for an example.

```
function ToggleButton() {
  // Step 1: Create a state variable to track the button's active state. (isActive, setIsActive). React.useState
  // Step 2: Function to toggle the button's active state
  return (
    <>
      <button
        className="btn"
        // Step 3: Add a click event handler to toggle the button's active state
        // Step 4: Use the state variable to conditionally render the button's text and the div below it
      >
        {/* {isActive ? 'ON' : 'OFF'} */ 'Practice'}
      </button>
      {/*<div>It may get difficult but with time you will always get better. This note is filled with gems - dig dont scr
    </>
  )
}
```

Practice

Introduction to React Hooks

Hooks are functions that let you "hook into" React state and lifecycle features from function components.



Why Hooks?

- Simpler Code: No more class components
- Reusable Logic: Share stateful logic between components
- Composition: Build complex features from simple ones
- Better TypeScript Support: More predictable types

Basic Hooks: useState & useEffect

useState

Syntax:

```
const [state, setState] = useState(initialValue)
```

```
function Counter() {
  const [count, setCount] = useState(0);
  const increment = () => setCount(count + 1)

  return ( ... );
}
```

Key Points:

- Returns [value, setter] array
- Initial value only used on first render
- State updates trigger re-renders
- Always call at top level of component

useEffect

Syntax:

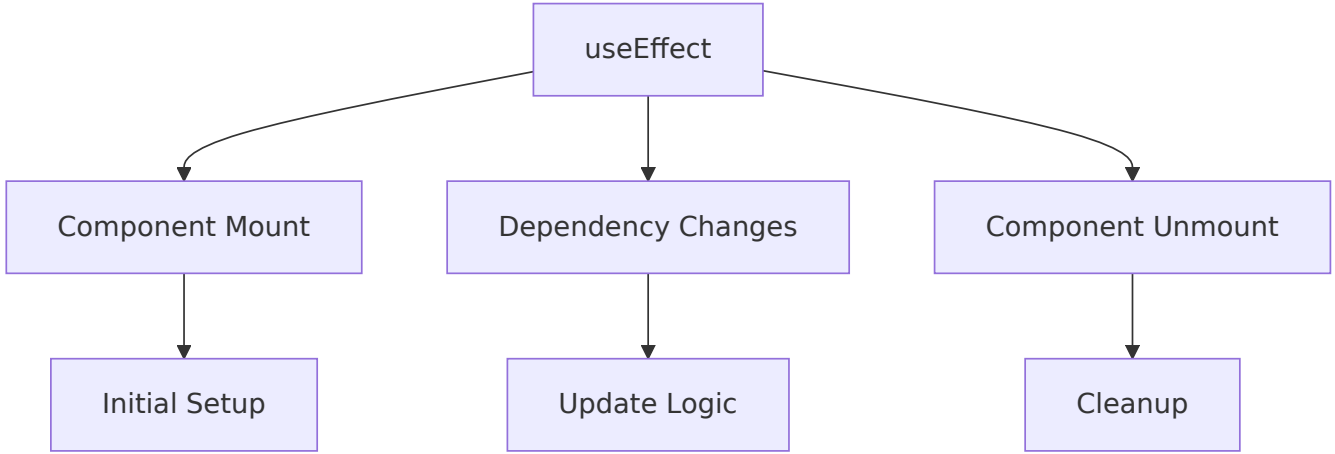
```
useEffect(setup, dependencies?)
```

```
function DataFetcher({ id }) {
  const [data, setData] = useState(null)
  useEffect(() => {
    let mounted = true // 1. Effect Setup
    async function fetchData() {
      const result = await fetch(`https://api.github.cc
      const json = await result.json()
      setData(json)
    }
    if (mounted) fetchData()
    return () => mounted = false // 2. Effect Cleanup
  }, [id]) // 3. Dependencies
  return <div>{data ? <DisplayData data={data} /> : 'Lo
}
```

Dependency Patterns:

- `[]` - Run once on mount
- `[dep1, dep2]` - Run when deps change
- No array - Run after every render
- Return function for cleanup

useEffect Hook



```
function DataFetcher({ id }) {
  const [data, setData] = useState(null)

  useEffect(() => {
    // 1. Effect Setup
    let mounted = false

    async function fetchData() {
      const result = await fetch(`https://api.github.com/users/${id}`)
      const json = await result.json()

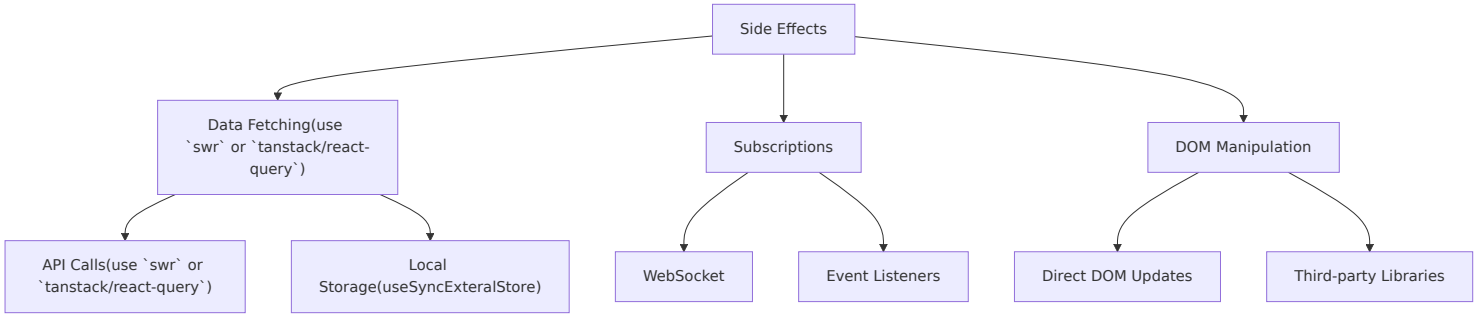
      if (!mounted) {
        setData(json)
      }
    }

    fetchData()

    // 2. Effect Cleanup
    return () => {
      mounted = true
    }
  }, [id]) // 3. Dependencies

  return <div>{data ? <DisplayData data={data} /> : 'Loading...'}</div>
}
```

Managing Side Effects



```
// 1. Data Fetching
function UserProfile({ userId }) {
  useEffect(() => {
    // Only fetch when userId changes, add a variable like ignore or mount to avoid race condition
    fetchUser(userId).then(setUser)
  }, [userId])
}

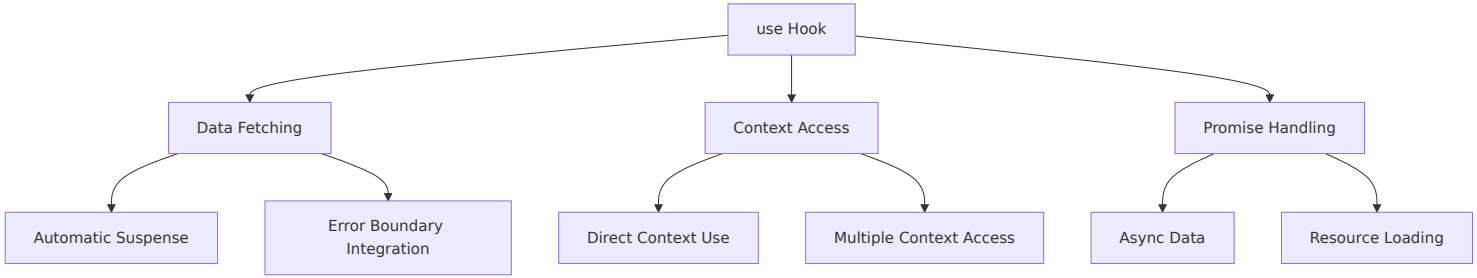
// 2. Event Listeners
function WindowSize() {
  useEffect(() => {
    const handleResize = () => {
      // Update size state
    }

    window.addEventListener('resize', handleResize)
    return () => window.removeEventListener('resize', handleResize)
  }, []) // Empty deps = only on mount/unmount
}

// 3. Subscriptions
function LiveData() {
  useEffect(() => {
    const subscription = dataSource.subscribe(handleData)
    return () => subscription.unsubscribe()
  }, [dataSource])
}
```

The New `use()` Hook: A Game Changer

The versatile new `use()` hook replaces multiple hooks, such as useEffect for data fetching as well as useContext and useState for consuming context data. It simplifies your code by handling promises and context with a single, elegant solution.



Fetching Data and Context Usage

```
import { createContext, use } from 'react'

const ThemeContext = createContext()

const ThemeConsumer = () => {
  const { theme, toggleTheme } = use(ThemeContext)

  return <button onClick={toggleTheme}>Current theme: {theme}</button>
}
```

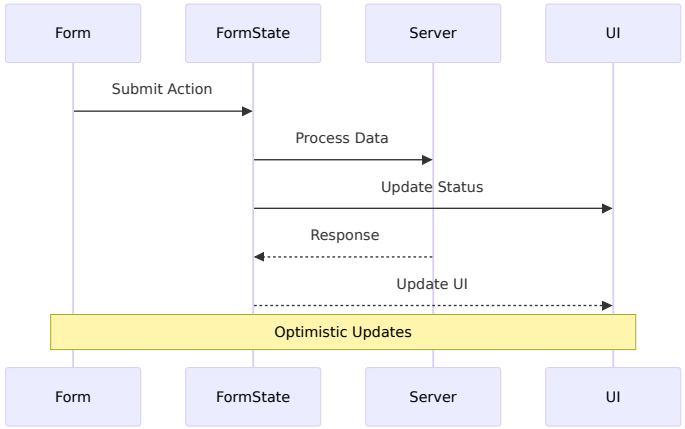
Form Handling Hooks

The `useFormStatus()` hook (React 19)

This hook provides real-time information about form submission status.

`Syntax`

```
const status = useFormStatus()
```



Basic Form Status Example

Simple form demonstrating useFormStatus hook for loading states

Example 1 of 2

```jsx
import { useFormStatus } from 'react-dom';

// Submit Button Component with Loading State
function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button
      disabled={pending}
      className="px-4 py-2 bg-blue-500 text-white rounded"
    >
      {pending ? 'Submitting...' : 'Submit'}
```

`useActionState()`

This hook manages form state and updates based on submission results.

```
import { useActionState } from 'react'

function Form() {
  const handleSubmit = async (prevState, formData) => {
    const username = formData.get('username')
    return username === 'admin'
      ? { success: true, message: 'Welcome!' }
      : { success: false, message: 'Invalid username' }
  }

  const [state, formAction] = useActionState(handleSubmit, null)

  return (
    <form action={formAction}>
      <input type="text" name="username" />
      <button type="submit">Login</button>
      {state && <p>{state.message}</p>}
    </form>
  )
}
```

Create a search form with loading state:

```
function SearchForm() {
  // Implement form state and status
  return (
    <form>
      {/* Add search input and submit button */}
      <br />
    </form>
  );
}
```

Show Solution

Complex State Management | Updating Objects and Arrays (Updating Objects and Arrays)

When working with complex state, such as objects and arrays, it's important to avoid mutating the state directly. Instead, you should create new objects and arrays using the spread operator (...) or `Array.prototype.map()` and `Array.prototype.filter()`.

Updating Objects:

```
const [person, setPerson] = useState({ name: 'John', age: 30 })

// Correct way to update an object
setPerson({ ...person, age: 31 })
```

Updating Arrays:

```
const [items, setItems] = useState([1, 2, 3])

// Correct way to update an array
setItems([...items, 4])
setItems(items.filter((item) => item =/= 2))
```

Maintaining immutability in your state updates is crucial for React's efficient re-rendering and comparison of previous and current state.

Complex State Updates

```
function FormWithComplexState() {
  const [form, setForm] = useState({
    user: { name: '', email: '', },
    preferences: { newsletter: false, theme: 'light', },
  })
  // Updating nested state
  const updateName = (name) => {
    setForm((prev) => ({
      ...prev,
      user: { ...prev.user, name, },
    }))
  }
  // Using state updater function
  const toggleNewsletter = () => {
    setForm((prev) => ({
      ...prev,
      preferences: {
        ...prev.preferences,
        newsletter: !prev.preferences.newsletter,
      },
    }))
  }
}
```

Immutability

Immutability refers to the concept of not modifying an object or array directly, but rather creating a new one with the desired changes. This is an important principle in React, as it allows React to efficiently determine what has changed between renders and optimize the rendering process. When you update state in React, you should always create a new object or array instead of modifying the existing one. This ensures that React can effectively compare the previous and current state and update the DOM efficiently.

Understanding Immutability in React

Operation	❌ Mutable (Wrong)	✅ Immutable (Correct)	Explana
Update Object Property	`person.age = 31;`	`setPerson({ ...person, age: 31 });`	Creates a object wit updated a property v keeping ot properties unchange
Add Array Item	`items.push(4);`	`setItems([...items, 4]);`	Creates a array with existing it plus the n item
Remove Array Item	`items.splice(index, 1);`	`setItems(items.filter(item => item !== 2));`	Creates a array excluding specified i

Example

```
function TodoList() {
  const [todos, setTodos] = useState([]);

  // ❌ Wrong way (mutation)
  const addTodoWrong = (text) => {
    todos.push({ text, completed: false });
    setTodos(todos);
  };

  return (/* JSX */);
}
```

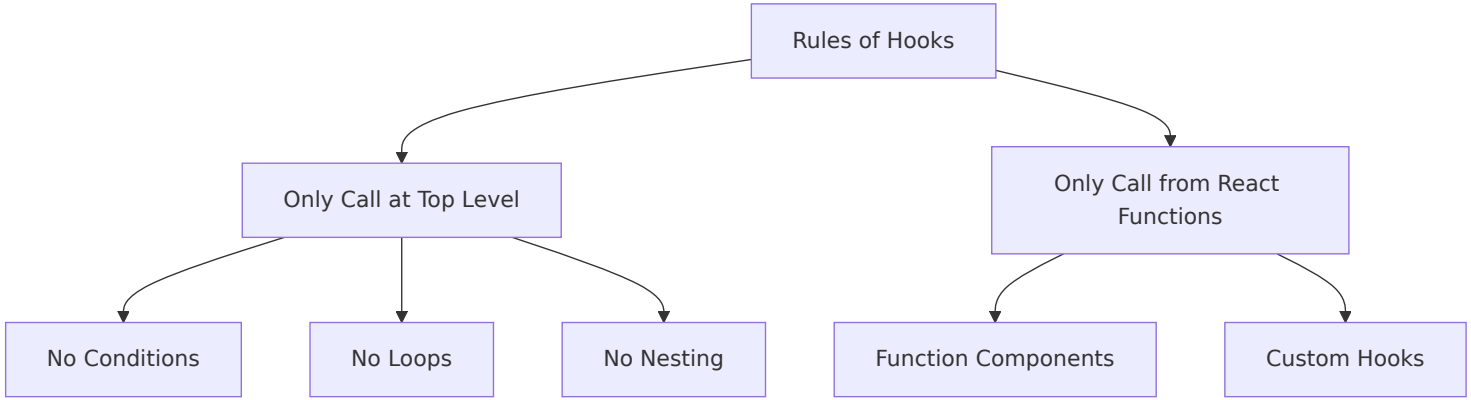
```
function TodoList() {
  const [todos, setTodos] = useState([]);

  // ✅ Right way (immutable)
  const addTodoRight = (text) => {
    setTodos([...todos, { text, completed: false }]);
  };

  // ✅ Updating nested objects
  const updateTodo = (id, updates) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, ...updates } : todo
    ));
  };

  return (/* JSX */);
}
```

Rules Of Hook



Rule 1: Only Call Hooks at the Top Level

✅ Good: Top level

```
const [count, setCount] = useState(0);
```

❌ Bad: Conditional hook

```
if (count > 0) {
  const [error, setError] = useState(null);
}
```

Create a form with complex state handling

```
// Create a form with complex state handling
function RegistrationForm() {
  // 1. Complex state structure
  const [form, setForm] = useState({
    personal: {
      firstName: '',
      lastName: '',
      email: '',
    },
    address: {
      street: '',
      city: '',
      country: '',
    },
    preferences: {
```

Assignment

1. Using the websocket provision in the API, use effect hook to create a real time display of `tasks` created, updated and delete. You job is to create a simple react application to show the realtime display of the socket communication. See this as an example Tasks Socket Client.

2. Implement a custom hook for this socket communication.

```
function LiveData() {
  class SimpleEventEmitter {
    constructor() {
      this.events = {};
    }

    on(event, listener) {
      if (!this.events[event]) {
        this.events[event] = [];
      }
      this.events[event].push(listener);
    }

    off(event, listener) {
      if (!this.events[event]) return;
      this.events[event] = this.events[event].filter(l => l !== listener);
    }

    emit(event, ...args) {
      if (!this.events[event]) return;
      this.events[event].forEach(listener => listener(...args));
    }
  }
```

```
state:2
error:
```

```
/**
 * An Async Generator that acts as our data source.
 * It manages its own internal state (num) and timing loop.
 */

function LiveData() {
  const [data, setData] = React.useState(undefined);
  const [error, setError] = React.useState(undefined);

  async function* liveDataGenerator() {
    let num = 0;

    while (true) {
      // 1. Pause execution for 1 second (simulating network/compute latency)
      await new Promise((resolve) => setTimeout(resolve, 1000));

      // 2. Update internal state
      num++;

      // 3. Determine what to yield based on the logic
      // We return an object distinguishing data from errors to keep the stream alive.
      // (Throwing an actual Error would terminate the generator).
      if (num % 3 === 0 && num > 1) {
```

```
state:2
error:
```

Managing State imperative vs declarative

Lifting State Up ⤴

Managing State

- Imperative vs Declarative
- How to structure state
- Lifting state up
- Reducer function
- Controlled vs Uncontrolled Components
- Keeping your components pure

Imperative vs Declarative

> Info
>
> Imperative: How to do something, Declarative: What to do

UI is a function of state and it should be declarative using JSX and the power of event interactivity in React. Identifying your component's different visual states, determine what triggers those state changes, Represent the state in memory using `useState`. Remove any non-essential state variables and connect the event handlers to set the state.

Reacting to user input is a key part of building interactive UIs. Handling user input with React typically involves updating the state of your components based on the user's input and interactions. Triggering state changes based on user input is how you build interactive UIs.

When a user types something into the form, the `submit` button becomes enabled. When the user clicks the `submit` button, both the form and the button becomes disabled and a spinner appears. If the network request is successful, the spinner disappears and a success message appears. If the network request fails, the spinner disappears and an error message appears. The above can be represented imperatively or declaratively. React enforces a declarative approach.

> **Tip**
>
> This involves listing out all the states your component can be in and how it transitions between those states. EMPTY: form has a disabled submit button, TYPING: Form has an enabled submit button, SUBMITTING: Form and button are disabled, spinner appears, SUCCESS: Spinner disappears, success message appears, ERROR: Spinner disappears, error message appears.

Declarative programming means describing the UI for each visual state rather than micromanaging the UI (imperative).

- Identify all its visual states
- Determine what triggers those state changes
- Represent the state in memory using `useState`
- Connect the event handlers to set the state

The example in the next slide shows a form that has a textarea input field and a submit button. The submit button is disabled when the textarea is empty. When the user types something into the textarea, the submit button becomes enabled. When the user clicks the submit button, the form and the button becomes disabled and a spinner appears. If the network request is successful, the spinner disappears and a success message appears. If the network request fails, the spinner disappears and an error message appears.

```
function Form() {
  const [answer, setAnswer] = React.useState('')
  const [error, setError] = React.useState(null)
  const [status, setStatus] = React.useState('idle')

  if (status === 'success') {
    return <h1 tabIndex="-1">That's right!</h1>
  }

  async function handleSubmit(e) {
    e.preventDefault()
    setStatus('submitting')
    try {
      await submitForm(answer)
      setStatus('success')
    } catch (err) {
      setStatus('error')
      setError(err)
    }
```

Sample quiz - idle

What should be the correct value?

Type your answer here ...

Submit

How to structure state

- Identify all the states your component can be in and how it transitions between those states.
- Group related states together.
- Avoid contradicting states.
- Avoid unnecessary and state duplication states.
- Avoid deeply nested states.

The goal behind these principles is to make state easy to update without introducing mistakes. Removing redundant and duplicate data from state helps ensure that all its pieces stay in sync.

> Info
>
> The more states you have, the more complex your component becomes. The more complex your component becomes, the more likely it is to have bugs.
> “Make your state as simple as it can be—but no simpler.” - Albert Einstein

```
const [x, setX] = useState(0)
const [y, setY] = useState(0)
```

```
const [position, setPosition] = useState({ x: 0, y: 0 })
```

Avoid unnecessary and state duplication states

```
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
-
-const [fullName, setFullName] = useState(firstName +
```

```
const [firstName, setFirstName] = useState('');
const [lastName, setLastName] = useState('');

+const fullName = firstName + ' ' + lastName;
```

Use spread operator to override the change part of the state when updating the state.

```
const [user, setUser] = useState({ firstName: '', lastName: '' })
setUser({ ...user, firstName: 'John' })
```

This way, you don't have to worry about the state being out of sync and unnecessary nesting of states. Keeping the states flat and simple as possible.

Lift state up

When several components need to reflect the same changing data, it's recommended to lift the shared state up to their closest common ancestor. That means if two child components share the same state, the state should be lifted up to their parent component.

```
function Parent() {
  const [count, setCount] = useState(0)
  return (
    <>
      <Child count={count} setCount={setCount} />
      <Child count={count} setCount={setCount} />
    </>
  )
}
```

The Child component can now be a functional component that receives the `count` and `setCount` as props and can update the state in the parent component. This is called lifting state up. Sharing state between components is a common pattern in React.

```jsx
function Panel({ title, children, isActive, onShow }) {
  return (
    <section className="panel">
      <h3>{title}</h3>
      {isActive ? (
        <p>{children}</p>
      ) : (
        <button className="btn" onClick={onShow}>
          Show
        </button>
      )}
    </section>
  )
}
```

```jsx
function Accordion() {
  const [activeIndex, setActiveIndex] = React.useState(0)
  return (
    <>
      <h2>Coding Schools</h2>
      <Panel
        title="AltSchool Africa"
        isActive={activeIndex === 0}
        onShow={() => setActiveIndex(0)}
      >
        Pushing to be the institution in Africa and the worl
        AltSchool Africa, we believe in the power of educati
        world.
      </Panel>
      <Panel
        title="Others"
        isActive={activeIndex === 1}
        onShow={() => setActiveIndex(1)}
      >
        Other coding schools in Africa are doing great thing
        pace and match the quality of education we provide a
      </Panel>
    </>
  )
}
```

Coding Schools

AltSchool Africa

Pushing to be the institution in Africa and the world at large. Here at AltSchool Africa, we believe in the power of education to change the world.

Others

Show

Reducer function

A reducer function is a function that takes the current state and an action and returns a new state. It's a way to manage state in a more predictable way. It's a pattern that's used in many programming languages and libraries. Its often useful when you have complex state logic that involves multiple sub-states or when the next state depends on the previous state. Extracting the state logic into a reducer function can make your code easier to understand and maintain.

```
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 }
    case 'decrement':
      return { count: state.count - 1 }
    default:
      throw new Error()
  }
}
```

```
function Counter() {
  const [state, dispatch] = React.useReducer(reducer, { coun
  return (
    <>
      Count: {state.count}
      <button onClick={() => dispatch({ type: 'increment' })
      <button onClick={() => dispatch({ type: 'decrement' })
    </>
  )
}
```

```
function Form() {
  function statusReducer(state, action) {
    switch (action.type) {
      case 'idle':
        return { status: 'idle', answer: '', error: null }
      case 'typing':
        return { ...state, status: 'typing', answer: action.answer, error: null }
      case 'submitting':
        return { ...state, status: 'submitting' }
      case 'success':
        return { status: 'success', answer: state.answer, error: null }
      case 'error':
        return { ...state, status: 'typing', error: action.error }
      default:
        return state
    }
  }

  const [state, dispatch] = React.useReducer(statusReducer, {
```

Sample quiz - idle

What should be the correct value?

Type your answer here...

Submit

Controlled vs Uncontrolled Components

Controlled components are components that render form elements whose values are controlled by React. Uncontrolled components are components that render form elements whose values are controlled by the DOM.

Controlled components are more flexible because you can control the value of the input field and the value of the input field can be derived from the state. Uncontrolled components are less flexible because you can't control the value of the input field and the value of the input field can't be derived from the state.

```
function ControlledInput() {
  const [value, setValue] = React.useState('')
  return (
    <input
      type="text"
      value={value}
      onChange={(e) => setValue(e.target.value)}
    />
  )
}
```

```
function UncontrolledInput() {
  const inputRef = React.useRef()
  return <input type="text" ref={inputRef} />
}
```

```jsx
function UncontrolledForm() {
  const nameRef = React.useRef(null)
  const emailRef = React.useRef(null)

  function handleSubmit(event) {
    event.preventDefault()

    // Access values directly from the DOM elements
    const name = nameRef.current.value
    const email = emailRef.current.value

    console.log('Name:', name)
    console.log('Email:', email)

    alert('Form submitted successfully!')

    // Optionally clear the fields
    nameRef.current.value = ''
    emailRef.current.value = ''
```

Name:

Email:

Submit

Keeping your components pure

A pure component is a component that renders the same output for the same input. It doesn't have any side effects and doesn't depend on anything other than its props and state. Pure components are easier to reason about and test because they don't have any side effects.

```
function PureComponent({ value }) {
  return <div>{value}</div>
}
```

```
function ImpureComponent() {
  return <div>{new Date().toLocaleTimeString()}</div>
}
```

Keeping your components pure make rendering predictable. Its easier to manage, it doesn't have any side effects and doesn't depend on anything other than its props and state. Effects are a way to add side effects to your components and it should be done with caution using events, refs and special hooks designed for such purpose.

> ⚠ Danger
>
> You should not mutate any of the inputs that your components use for rendering. That includes props, state, and context. To update the screen, "set" state instead of mutating preexisting objects.
> Strive to express your component's logic in the JSX you return. When you need to "change things", you'll usually want to do it in an event handler. As a last resort, you can useEffect.

Data Binding controls, validations

Handling Form Inputs ⌨

Form

- Introduction
- Handling Form Inputs - Data Binding
- Form Controls
- Form Validation
- React Hook Form
- Tanstack Form
- New Form Features and Updates in React 19

Introduction

Why are forms so important ?

Forms allow users to input and submit information. They enable users to perform essential tasks such as sharing data, completing actions, and providing feedback. Without forms, core functionalities on the web like logging in or signing up won't be possible. Forms are what facilitates user interaction with the page , enabling dynamic experiences.

The form here will submit then the page will refresh, but this is not what we want to happen. We want to prevent this default behavior and let `React control the form and its data`. Handling forms is about how you handle the data when it changes value or gets submitted.

How can I collect user input and handle it effectively in React ?

```jsx
import React, { useState } from 'react'

function App() {
  const [name, setName] = useState('')

  const handleFormSubmit = () => {
    console.log('Submitted Name:', name)
  }

  return (
    <div>
      <form onSubmit={handleFormSubmit}>
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
          required
        />
        <button type="submit">Submit</button>
      </form>
    </div>
  )
}
```

Handling Form Inputs - Data Binding

When users interact with form elements (like inputs), it may trigger changes in the data and UI. This interaction is managed through data binding — how UI element reacts when the `data changes` and vice versa.

There are two types of data binding in React:

- One-way data binding: Data flows in a single direction, such as from a parent component to a child via props or from the component's state to the UI. React favors one-way binding by default because it keeps components predictable and easy to debug.
- Two-way data binding: Data flows both ways between UI elements (like inputs) and the component's state. When the user interacts with a form element, it updates the state, and changes in the state trigger re-renders of the UI. This is typically achieved using controlled components (e.g., input, textarea, select).

By default, `React supports one-way data binding`, where data flows in one direction—from the component's state to the UI. In this setup, changes to the data update the UI, but changes made directly in the UI (like typing in an input field) will not update the state unless explicitly managed.

```
import React from 'react'

function App() {
  const inputValue = 'Hello'

  return (
    <div>
      <input type="text" value={inputValue} />
      <p>Value: {inputValue}</p>
    </div>
  )
}

export default App
```

In this example above, if you try changing the value "Hello" directly in the input field, it won't update because React only supports one-way data binding by default.

However, React allows two-way data binding using `controlled components`. This is especially useful for managing form elements like input, textarea, and select, where the values are synchronized with the component's state. Controlled components ensure the `UI and state stay in sync` making it easier to handle user input and dynamic form behavior.

Uncontrolled Components: One-Way Data Binding

In uncontrolled components, form values are managed directly by the DOM, and React accesses the values through refs. This approach suits simple forms where performance and minimal re-renders matter.

```
import React, { useRef } from 'react'

function App() {
  const inputRef = useRef(null)

  const handleSubmit = () => {
    alert(`Input Value: ${inputRef.current.value}`)
  }

  return (
    <div>
      <input type="text" ref={inputRef} />
      <button onClick={handleSubmit}>Submit</button>
    </div>
  )
}
export default App
```

The `inputRef` allows access to the input field directly in the DOM. Clicking the 'Submit' button retrieves the current value without React managing every input change.

Controlled Components: Two-Way Data Binding

In controlled components, form input values are linked to state, ensuring the input and state are always in sync.

```
import React, { useState } from 'react'

function App() {
  const [inputValue, setInputValue] = useState('Hello')

  const handleChange = (event) => setInputValue(event.target.value)

  return (
    <div>
      <input type="text" value={inputValue} onChange={handleChange} />
      <p>Value: {inputValue}</p>
    </div>
  )
}
export default App
```

The `inputValue` state stores the input field's value. The onChange event updates the state with every input change, ensuring two-way synchronization.

Form Controls

Form controls refer to input elements like text fields, checkboxes, radio buttons, and dropdowns. Here's how to handle different controls in React.

✔ Text Input

```
import React, { useState } from 'react'

function TextInput() {
  const [name, setName] = useState('')

  const handleChange = (e) => {
    setName(e.target.value)
    console.log('My name is:', e.target.value)
  }

  return (
    <input
      type="text"
      value={name}
      onChange={handleChange}
      placeholder="Enter your name"
    />
  )
}
```

✔ Checkbox

```
import React, { useState } from 'react'

function CheckboxInput() {
  const [isChecked, setIsChecked] = useState(false)
  const handleChange = (e) => {
    setIsChecked(e.target.checked)
    console.log('Accepted terms and conditions?:', e.target
  }
  return (
    <label>
      <input type="checkbox" checked={isChecked} onChange={h
      Accept Terms & Conditions
    </label>
  )
}

export default CheckboxInput
```

✔ Radio Buttons

```jsx
import React, { useState } from 'react'

function RadioButton() {
  const [gender, setGender] = useState('')
  const handleChange = (e) => {
    setGender(e.target.value)
    console.log('Radio Button:', e.target.value)
  }

  return (
    <>
      <label>
        <input
          type="radio"
          name="gender"
          value="male"
          checked={gender === 'male'}
          onChange={handleChange}
        />{' '}
        Male
      </label>
      <label>
        <input
          type="radio"
          name="gender"
          value="female"
```

✔ Select (Dropdown)

```jsx
import React, { useState } from 'react'

function SelectInput() {
  const [fruit, setFruit] = useState('')

  const handleChange = (e) => {
    setFruit(e.target.value)
    console.log('Selected Fruit:', e.target.value)
  }

  return (
    <select value={fruit} onChange={handleChange}>
      <option value="">Select a fruit</option>
      <option value="apple">Apple</option>
      <option value="banana">Banana</option>
      <option value="orange">Orange</option>
    </select>
  )
}

export default SelectInput
```

Form Validation

Validation ensures that the input meets specific requirements. It can be done manually or using libraries.

- Manual Validation

```
import React, { useState } from 'react'

function EmailValidation() {
  const [email, setEmail] = useState('')
  const [error, setError] = useState('')
  const validateEmail = (value) => {
    if (!value) {
      setError('Email is required')
    } else if (value.length < 5) {
      setError('Email must be at least 5 characters long')
    } else {
      setError('')
    }
  }

  const handleChange = (e) => {
    const { value } = e.target
    setEmail(value)
    validateEmail(value)
```

```
const handleSubmit = (e) => {
  e.preventDefault()
  if (!error) {
    console.log('Email submitted:', email)
  }
}

return (
  <form onSubmit={handleSubmit}>
    <input type="email" value={email} onChange={handleChange} placeh
    {error && <p style={{ color: 'red' }}>{error}</p>}
    <button type="submit" disabled={!!error}>
      Submit
    </button>
  </form>
)
}

export default EmailValidation
```

Validation using libraries - Zod

Zod is a schema validation library that works well with `TypeScript` and `React`. Zod is particularly useful for validating data from forms thereby helping to catch errors early in the development process.

To use Zod in a React application, you'll first need to install it: `npm/pnpm install zod`

Step-by-step example of how to use Zod for validating a user registration form.

1 Define a Schema: Create a Zod schema for the data you want to validate.

```
import { z } from 'zod'

// Define the schema for the registration form
const registrationSchema = z.object({
  username: z .string() .min(3, { message: 'Username must be at least 3 characters long' }),
  email: z.string().email({ message: 'Invalid email address' }),
  password: z .string() .min(6, { message: 'Password must be at least 6 characters long' }),
})
```

2 Validate Data: Use the schema to validate form data and handle errors appropriately.

React Hook Form

React Hook Form is a powerful library for managing form state with minimal re-renders and providing better performance for complex forms. It manages form state, handles form submission, and provides basic validation support.

To use React Hook Form in a React application, you'll first need to install it with @hookform/resolvers:

```
npm/pnpm install react-hook-form @hookform/resolvers
```

Why add @hookform/resolvers?

It is a companion package that connects external schema validation libraries (like Zod) with React Hook Form.

⚠ Using only Zod, you manually handle input state, errors, and validation for each field, updating state on every change. This can become complex in larger forms as you write custom code to manage the form state, errors, and re-renders. However, React Hook Form centralizes state management for the form. You get optimized re-renders, reduced code for managing state and errors, and a streamlined way to track form fields. It also supports complex forms with nested fields and conditional validation out of the box.

The same Registration form using React-Hook-Form and Zod is on the next page ➠



Click to load iframe

https://codesandbox.io/embed/26sv4h?view=editor+%2B+preview&module=%2Fsrc%2FApp.js%3A3%2C64

Tanstack `<Form />`

TanStack Form is the ultimate solution for handling forms in web applications, providing a powerful and flexible approach to form management. Designed with first-class TypeScript support, headless UI components, and a framework-agnostic design, it streamlines form handling and ensures a seamless experience across various front-end frameworks.

To use Tanstack Form in a React application, you'll first need to install it: `npm/pnpm/bun/yarn install @tanstack/react-form`

- A Form Instance is an object that represents an individual form and provides methods and properties for working with the form.
- A Form Field is a component that represents an individual field within a form. It provides methods and properties for managing the field's state, validation, and submission.
- Each field has its own state, which includes its current value, validation status, error messages, and other metadata. You can access a field's state using the field.state property.

```
import React from 'react';
import { useForm } from '@tanstack/react-form';

export default function App() {
  const form = useForm({
    defaultValues: { firstName: '', lastName: '', },
    onSubmit: async ({ value }) => { // Do something with form data console.log(value); },
  });

  return (
    <form onSubmit={(e) => { e.preventDefault(); e.stopPropagation(); form.handleSubmit(); }} >
      <div>
        <form.Field name="firstName"
          validators={{
            onChange: ({ value }) => !value ? 'A first name is required' : value.length < 3 ? 'First name must be at least
            onChangeAsyncDebounceMs: 500,
            onChangeAsync: async ({ value }) => {
              await new Promise((resolve) => setTimeout(resolve, 1000));
              return ( value.includes('error') && 'No "error" allowed in first name' );
            },
          }}
          children={(field) => {
            return (
              <>
                <label htmlFor={field.name}>First Name:</label>
                <input id={field.name} name={field.name} value={field.state.value} onBlur={field.handleBlur} onChange={(e)
                {!field.state.meta.isValid && ( <em style={{color: red;}} role="alert">{field.state.meta.errors.join(', ')}
```



Click to load iframe

https://stackblitz.com/edit/tanstack-form-ebyuvxlk?ctl=1&embed=1&file=src%2Findex.jsx&hideExplorer=1

Validation with Standard Schema Libraries

TanStack Form supports validation with standard schema libraries like Zod, Valibot, and Arktype. This allows you to define your validation rules using the library of your choice and integrate them seamlessly into your form handling process.

```
import { z } from 'zod'

const userSchema = z.object({
  age: z.number().gte(13, 'You must be 13 to make an account'),
})

function App() {
  const form = useForm({
    defaultValues: { age: 0 },
    validators: { onChange: userSchema },
  })
  return (
    <div>
      <form.Field
        name="age"
        children={(field) => {
          return <>{/* ... */}</>
        }}
      />
    </div>
```

New Form Features and Updates in React 19

React 19 introduces powerful new features to simplify form handling, streamline server interactions, and enhance accessibility in web applications. These updates reflect a focus on efficient form processing, enabling developers to manage complex user inputs with less code and more intuitive design patterns.

These features collectively empower developers to build faster, more accessible, and user-friendly applications by making form processing easier and more robust.

We'll explore two of the key form-handling features in React 19:

1 `Form Actions` — Simplified methods for handling form submissions directly to the server, reducing the need for complex API routes and custom handlers.

2 `Form Reset and State Management` — Improved control over form reset actions and state updates, allowing developers to manage form data more efficiently.

`1. Form Actions`

In previous React versions, handling form submissions typically involved writing functions to process form data within the component. You often had to rely on custom handlers and libraries (like React Hook Form) for complex form logic. Additionally, any server interactions had to be handled through API routes, adding an extra layer of complexity.

Form Actions in React 19 introduce a more streamlined approach. They are a specialized implementation of `Server Actions` (actions in React 19 that allow any component to perform server-side operations directly) that focus specifically on handling form submissions. It allow developers to attach specific actions directly to form submissions without needing intermediate API routes or external libraries. This feature integrates form handling with server-side logic seamlessly, enabling simpler, more direct interactions with the server.

It simplify this process by allowing us to handle submissions directly on the server side, bypassing the need for an API route. Form actions make it easier to define and customize actions like saving, editing, or deleting data right in the form itself, making the handling of form logic more straightforward and efficient.

See how forms were handled before form actions and how they're handled using form actions



Click to load iframe

https://stackblitz.com/edit/vitejs-vite-j9uc3f?ctl=1&embed=1&file=src%2FApp.jsx&hideExplorer=1&hideNavigation=1

`2. Form Reset and State Management`

React 19 introduced improvements in managing form state and resetting forms, making it easier to handle form data consistently and efficiently. Previously, managing form reset behavior required explicit handling of state and additional code. The update make this process more efficient and straightforward by making form resets more predictable and easier to manage, reducing the need for extra code and ensuring a cleaner, more intuitive form experience.

Before React 19, form resetting was done by manually updating each field's state or setting all state variables back to their initial values. This approach could lead to more verbose code and was prone to errors, especially in large forms but in React 19, the entire form state is managed more easily. The approach is particularly useful with Form Actions or any forms that use server-side actions because it resets the form to its original state automatically, without needing to define each field individually.

Key Benefits of React 19's Streamlined Form Reset and State Management

1. `Improved Simplicity`: Forms can be reset without manually updating each field's state.
2. `Enhanced Performance`: Reducing the reliance on multiple useState calls minimizes re-renders.
3. `Better Compatibility with Server Actions`: When using server actions for form submissions, form data is automatically managed and reset, keeping forms clean and ready for subsequent submissions.



Click to load iframe

https://codesandbox.io/embed/ypmc3c?view=editor+%2B+preview&module=%2Fsrc%2Fcomponents%2Fform.js%3A39%2C5-39%2C8

Assignment

Read more on Zod and @tanstack/react-form

Create a form using `@tanstack/react-form` that includes fields for name, age, and email. Utilize Zod to define a validation schema that ensures the name is a non-empty string, the age is a positive integer, and the email matches a valid email format. Implement the form such that when the user submits it, any validation errors are displayed beneath the corresponding input fields, guiding the user to correct their entries. Use the useForm hook from Tanstack Form to manage form state and error handling, and integrate `Zod's zodResolver` to link the validation schema with the form. Ensure that the form is `styled for clarity and usability`, enhancing the user experience during input validation.

Assignment

Build a simple `Product Creation Application`. Build a simple form that will create a product using the API product endpoint requirements. Add proper validation based on the API required fields. Use All the learning from the class teaching to guide you. A form library and a no form library version is required.

More Hooks useRef, useLayoutEffect

useReducer & useContext

More Hooks

- useRef
- useReducer
- useContext
- useTransition
- useLayoutEffect
- useDeferredValue
- useActionState
- useFormStatus
- useOptimistic
- customHooks
- useSyncExternalStore
- useEffectEvent

✴ useRef

useRef is a React Hook that lets you `reference a value that's not needed for rendering` - it is used to persist a mutable reference without causing unnecessary re-renders. It can:

- Access and manipulate DOM elements directly.
- Store a mutable value that doesn't trigger re-renders when updated.

Syntax

```
import { useRef } from 'react'

const refContainer = useRef(initialValue)
```

- Initialization: useRef `accepts a parameter (initialValue)` during the first render, which initializes the ref.current property. This initial value is ignored in subsequent renders.
- Returned Object: It returns an object with a single property, `current`.

Unlike useState, refs are mutable. You can directly update ref.current without causing a re-render.

Why useRef?

They are used for tracking previous state without triggering re-render.

Use `useRef` for managing data that doesn't trigger re-renders, such as storing interval IDs for animations or timers, tracking previous form input values, or caching API responses for debouncing.

It's also indispensable for DOM manipulations like focusing input fields, managing scroll positions for infinite scroll, detecting when elements come into view (e.g., lazy-loading images or videos), or controlling canvas elements in animations.

Additionally, it's useful for `integrating third-party libraries` like sliders, charts, or maps where direct DOM interaction is required.

```
const ref = useRef(initialValue)
console.log(ref.current) // Access the current value
ref.current = newValue // Update the value manually
```

⚠ Do not read or write `ref.current` during rendering, except for initialization.

useState vs useRef

Feature	`useState`	`useRef`
Triggers Re-render on Update	Yes	No
Mutable	No, immutable	Yes, mutable (`ref.current` can be changed)
Safe to Read During Render	Yes	No
Best Use Case	Displaying/updating UI	Storing data not affecting UI

On the next page , in the counter button implemented with `useState`, the screen updates to reflect the new count because changing the state triggers a re-render of the component. However, if the same counter is implemented with `useRef`, the count can be updated by mutating ref.current, but the component will not re-render, so you would never see the count change on the screen.

This highlights a key difference: `useState` is for reactive data that affects the UI, while `useRef` is for storing mutable data that doesn't influence rendering.



Click to load iframe

https://codesandbox.io/embed/566hdt?view=editor+%2B+preview&module=%2Fsrc%2Fcomponents%2FuseRefCounter.jsx&hidenavigation=1

✴ useReducer

useReducer is useful for managing complex state logic.

```
const [state, dispatch] = useReducer(reducer, initialState, init?)
```

The hook takes three parameters: a reducer function, an initialState, and an optional init function.

The `reducer` is a function that specifies how the state is updated, structured as `(state, action) ⇒ newState` (React passes the current state and action to the reducer , which calculates and returns the next state. React then saves the new state, re-renders your component with it, and updates the UI accordingly).

The `initialState` is the starting value of the state.

The optional `init` function allows you to compute the initial state, useful for heavy initialization logic.

useReducer returns an array with two elements: `state` and `dispatch`. The state holds the current value, while dispatch is a function that lets you change state in response to interaction - you can call to send actions, update the state, and trigger a re-render. This makes useReducer ideal for managing complex state logic or when state transitions depend on the current state and actions.

Use Cases: Complex state logic & Centralized state management within a component.

Using useReducer in a counter

```
import React, { useReducer } from 'react'

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 }
    case 'decrement':
      return { count: state.count - 1 }
    default:
      throw new Error()
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 })

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    </div>
  )
}
```

useState vs useReducer

Feature	`useState`	`useReducer`
Best for	Simple state logic, isolated values	Complex state logic with multiple transitions
State shape	Single or simple data types (e.g., numbers, booleans)	Complex objects or interdependent fields
Update	State setters (setState) for direct updates	Reducer function for controlled updates
Re-renders	On state updates via setter	On dispatch when the reducer returns a new state

You should use useReducer when your state logic becomes complex or involves multiple related variables. However, if your state is simple—like a single boolean toggle, a numeric counter, or isolated values—useState is more appropriate.



Click to load iframe

https://codesandbox.io/embed/vkmlhg?view=editor+%2B+preview&module=%2Fsrc%2Fcomponents%2FuseStateCart.jsx%3A25%2C54

Context & Performance Hooks

useContext

Syntax

`const value = useContext(SomeContext)`

```
const ThemeContext = createContext();

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <ThemedButton />
    </ThemeContext.Provider>
  );
}

function ThemedButton() {
  const theme = use(ThemeContext);
  return <button className={theme}>Click me</button>;
}
```

Use: Global state sharing without prop drilling.

useMemo

Syntax

`const memoized = useMemo(() => fn(), [deps])`

```
function ExpensiveComponent({ items, filter }) {
  const filteredItems = useMemo(() => {
    return items.filter(item => item.category === filter);
  }, [items, filter]);

  return (
    <div>
      {filteredItems.map(item => <div key={item.id}>{item.n
    </div>
  );
}
```

Use: Memoize expensive calculations to prevent unnecessary re-computations.



Click to load iframe

https://codesandbox.io/embed/zxzrcz?view=editor+%2B+preview&module=%2Fsrc%2Fcomponents%2FuseContext.jsx%3A41%2C29

```
import { createContext, useContext } from 'react'

const ThemeContext = createContext()

function ThemeProvider({ children }) {
  return <ThemeContext.Provider value="dark">{children}</ThemeContext.Provider>
}

function ThemedComponent() {
  const theme = use(ThemeContext); // replace with `use`
  return (
    <div style={{ background: theme === 'dark' ? '#333' : '#fff' }}>
      Themed Component
    </div>
  )
}

function App() {
  return (
    <ThemeProvider>
      <ThemedComponent />
    </ThemeProvider>
  )
}
```

Without useContext: You'd need to pass the theme prop down manually, making the code harder to maintain.

Concurrent Features

useTransition

```
const [isPending, startTransition] = useTransition()
```

```
function SearchApp() {
  const [isPending, startTransition] = useTransition();
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  const handleSearch = (newQuery) => {
    setQuery(newQuery);
    startTransition(() => {
      // Non-urgent update
      setResults(expensiveSearch(newQuery));
    });
  };
  return ( ... );
}
```

Use: Mark non-urgent updates to keep UI responsive.

startTransition

```
startTransition(() => { /* update */ })
```

```
import { startTransition } from 'react';

function FilterApp() {
  const [filter, setFilter] = useState('');
  const [filteredData, setFilteredData] = useState(data);

  const handleFilter = (newFilter) => {
    setFilter(newFilter);
    startTransition(() => {
      // Expensive filtering operation
      setFilteredData(data.filter(item => item.name.include
    });
  };
  return ();
}
```

Use: Wrap expensive updates to prevent blocking.

✴ useLayoutEffect

useLayoutEffect is a hook designed to perform synchronous side effects after DOM mutations `but before the browser repaints the screen`. This ensures that any layout measurements or updates are applied before the user sees them, making it valuable for scenarios where precise alignment or measurements of DOM elements are critical.

It is a version of useEffect. However, in contrast, useEffect runs asynchronously after the paint cycle, making it more suitable for non-blocking side effects like setting up subscriptions or setting up event listeners. While `both hooks allow for side effects`, their timing and use cases differ significantly.

Syntax

```
useLayoutEffect(setup, dependencies?)
```

Use Cases:

- DOM Measurements: If you need to measure an element's dimensions before rendering dependent content, such as calculating a tooltip's position.
- Avoiding Flickers: When you want to avoid the user seeing visual changes (e.g., moving elements) after the paint cycle.

useLayoutEffect vs useEffect

Use `useLayoutEffect` when you need to measure or synchronize changes to the DOM before the browser repaints, such as positioning tooltips or avoiding flickers in animations. For most other side effects, like fetching data or setting up subscriptions, prefer `useEffect` because it runs after the browser paint, ensuring better performance and smoother user experiences.

⚠ useLayoutEffect can hurt performance. Prefer useEffect when possible.

Feature	`useEffect`	`useLayoutEffect`
Timing	Runs *after* the browser paints the screen.	Runs *before* the browser repaints the screen.
⚡ Performance Impact	Non-blocking, better for most use cases.	Blocks rendering, may hurt performance.
Server Behavior	Executes normally on the server.	No effect on the server.



Click to load iframe

https://codesandbox.io/embed/36d7cr?view=editor+%2B+preview&module=%2Fsrc%2FApp.js%3A8%2C8-8%2C24

In the previous example , a tooltip was implemented using both useEffect and useLayoutEffect.

`Observations`:

- useEffect: On slower devices, you might notice a slight flicker when the tooltip adjusts its position. This happens because the position calculation occurs after the browser paints the DOM.
- useLayoutEffect: The tooltip renders in the correct position immediately, avoiding the flicker. This is because layout calculations and DOM updates are completed before the browser paints.

`Key Difference`:

Use useLayoutEffect when you need to measure DOM elements and apply layout changes before the browser paints. Use useEffect for non-blocking effects to maintain performance unless layout adjustments are strictly necessary.

✴ useDeferredValue

useDeferredValue is a React Hook that lets you `defer the update of certain parts of the user interface`. This improves application responsiveness by delaying non-urgent updates during resource-intensive operations & high-priority user interactions, like typing in a search box, filtering or rendering a large list.

Syntax

```
const deferredValue = useDeferredValue(value)
```

value: The original value to defer and it returns a deferred value that may lag behind the original during updates.

How it Works

1 On input change, React first renders with the previous deferred value.

2 React attempts to re-render with the updated deferred value in the background.

3 If background rendering suspends (e.g., waiting for data), the old deferred value remains visible until ready.

Use Cases:

- `Search Filtering`: When typing in a search bar that triggers heavy computations, you can use useDeferredValue to show intermediate results while keeping the typing experience snappy.
- `Rendering Large Data`: While dynamically rendering large datasets, deferring updates prevents blocking other user interactions.

Advantages

- `Enhanced Responsiveness`: Improves UI performance by prioritizing critical updates.
- `Ease of Integration`: Simple to add into existing React codebases for performance optimization.

⚠ Pitfall in useDeferredValue Optimization

When using useDeferredValue to optimize performance, a key pitfall is that components depending on the deferred value must avoid unnecessary re-renders. This often requires wrapping such components in `React.memo`.



Click to load iframe

https://codesandbox.io/embed/cc6xlm?view=editor+%2B+preview&module=%2Fsrc%2FApp.js%3A5%2C17

`useActionState`

Manages state for form actions and async operations. It provides pending state, error handling, and optimistic updates for form submissions and server actions. The was discussed in the previous chapter.

`useFormStatus`

Returns the status of form submissions, including whether a form is pending submission. This is particularly useful for showing loading states on submit buttons. The was discussed in the previous chapter.

`useOptimistic`

Enables optimistic UI updates by allowing you to show the expected result of an action immediately, then reconciling with the actual server response when it arrives. Allows you to optimistically update the UI while an asynchronous operation is in progress. It shows the expected result immediately and automatically reverts if the operation fails.

```
const [optimisticState, addOptimistic] = useOptimistic(state, updateFn);
```

More on `useOptimistic`

```
import { useOptimistic } from 'react';

function AppContainer() {
  const [optimisticState, addOptimistic] = useOptimistic(
    state,
    // updateFn
    (currentState, optimisticValue) => {
      // merge and return new state
      // with optimistic value
    }
  );
}
```

Concurrent Features

useTransition:

- Hook with pending state
- Built-in loading indicator
- Automatic priority handling

startTransition:

- Standalone function
- No loading state
- Manual priority control

Custom Hooks

Custom Hooks are reusable functions that contain logic shared across multiple components. They allow you to extract complex logic from components, making them more readable and maintainable. Custom Hooks follow the naming convention `useSomething` to indicate that they are hooks.

- UseForm
- UseFetch

As a way to practice, create a custom hook named `useToggle` and `useCounter`.



Click to load iframe

https://stackblitz.com/edit/vitejs-vite-fdk26g?ctl=1&embed=1&file=src%2Fhooks%2FuseForm.js



Click to load iframe

https://stackblitz.com/edit/vitejs-vite-hyv9j2?ctl=1&embed=1&file=src%2Fhooks%2FuseFetch.js

useToggle Custom Hook

```
function useToggle(initialValue = false) {
  const [value, setValue] = React.useState(initialValue);
  const toggle = React.useCallback(() => {
    setValue(prev => !prev);
  }, []);
  const setTrue = React.useCallback(() => {
    setValue(true);
  }, []);
  const setFalse = React.useCallback(() => {
    setValue(false);
  }, []);
  return [value, { toggle, setTrue, setFalse }];
}

function ToggleButton() {
  const [isOn, { toggle, setTrue, setFalse }] = useToggle(false);

  return (
    <div>
      <button onClick={toggle}>Toggle</button> <button onClick={setTrue}>ON</button>
      <button onClick={setFalse}>OFF</button>
      <span>Status: {isOn ? 'ON' : 'OFF'}</span>
    </div>
  );
}
```

useCounter Custom Hook

```
function useCounter(initialValue = 0, step = 1) {
  const [count, setCount] = React.useState(initialValue);

  const increment = React.useCallback(() => {
    setCount(prev => prev + step);
  }, [step]);
  const decrement = React.useCallback(() => {
    setCount(prev => prev - step);
  }, [step]);
  const reset = React.useCallback(() => {
    setCount(initialValue);
  }, [initialValue]);
  return [count, { increment, decrement, reset }];
}

function Counter() {
  const [count, { increment, decrement, reset }] = useCounter(0, 5);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>+{5}</button> <button onClick={decrement}>-{5}</button> <button onClick={reset}>Reset</button>
    </div>
  );
}
```

useSyncExternalStore: useOnlineStatus Hook

```
// Final: Complete implementation with all functions
function useOnlineStatus() {
  const isOnline = React.useSyncExternalStore( subscribe, getSnapshot );
  return isOnline;
}

function subscribe(callback) {
  window.addEventListener('online', callback);
  window.addEventListener('offline', callback);
  return () => {
    window.removeEventListener('online', callback);
    window.removeEventListener('offline', callback);
  };
}

function getSnapshot() {
  return navigator.onLine;
}

function App() {
  const isOnline = useOnlineStatus();
  return <div>{isOnline ? ' Online' : ' Offline'}</div>
}
```

Understanding useSyncExternalStore

Why useSyncExternalStore?

- Handles external store subscriptions safely
- Prevents tearing (inconsistent state) during concurrent rendering
- Automatically manages subscription lifecycle
- Works with React's concurrent features

Key Functions:

1. `subscribe(callback)`:
- Registers callback for store changes
- Returns cleanup function
- Called when component mounts/updates

2. `getSnapshot()`:
- Returns current store value
- Must be synchronous
- Called during render to get current state

Use Cases:

- Browser APIs (online status, geolocation, media queries)
- Third-party state management libraries
- WebSocket connections
- Browser storage (localStorage, sessionStorage)

useEffectEvent

- Extract non-reactive logic from Effects into an Effect Event; it always reads the latest props/state without retriggering the Effect.
- Define with `useEffectEvent(fn)` at the top level; call only inside Effects (`useEffect`, `useLayoutEffect`, `useInsertionEffect`).
- Use for non-reactive work (logging, telemetry, imperative calls) instead of making the Effect depend on extra values.

```
1  import { useEffect, useEffectEvent } from 'react'
2
3  function ChatRoom({ roomId, theme }) {
4    const onConnected = useEffectEvent(() => {
5      showNotification('Connected!', theme) // theme stays fresh, not reactive
6    })
7
8    useEffect(() => {
9      const connection = createConnection(roomId)
10     connection.on('connected', () => onConnected())
11     connection.connect()
```

Effect Events aren't a dependency shortcut—keep real reactive deps in the array.

Routes & Links redirects, protecting routes

Navigating Between Pages

Routing and Navigation

- Introduction to Routing and React Router
- Setting Up Routes and Links
- Navigating Between Pages
- Dynamic Routing and URL Parameters
- Redirects and Protected Routes
- Tanstack Router
- Frameworks
- SEO with unhead

What is Routing

In traditional websites, navigating to a new page involves the browser requesting a new HTML, CSS and JS document from the server, resulting in a full page reload. This approach is often slow and disrupts the user experience. With routing in `SPAs (Single Page Applications)`, we can simulate a multi-page experience without reloading the page, creating a faster, smoother experience.

How does routing work

When routing is handled on the client side (in the browser), JavaScript manages which content to display based on the URL. Instead of fetching new HTML files for each view, a single-page application (SPA) `dynamically updates displayed components`, creating the illusion of navigating multiple pages within a single web page.

This is where React Router comes in. As the most popular library for navigation in React applications, React Router enables seamless route-based navigation in SPAs. `React Router enables "client side routing"`. It allows developers to define routes that update views without reloading the page, giving users a smooth, app-like experience as they move between different "pages" within the application.

Routing Comparison: Traditional Web vs. Client-Side Routing

Step	Traditional Web Routing	Client-Side (SPA) Routing
1. Home Page	User visits `/`, browser loads HTML, CSS and JS from the server	User visits `/`, React Router loads the Home component without reload
2. Navigation to About	New request to the server for `/about`	React Router intercepts `/about` and updates view instantly
3. Content Update	Server sends new HTML, CSS and JS for `/about`, replacing the page	"About" component rendered dynamically within the page
4. User Experience	Full-page reloads cause delays	Navigation is smooth and instant

Setting Up Routes and Links

To use React Router, first install the react-router package: `npm/pnpm/bun install react-router`

After installing React Router, you can start defining routes and setting up navigation. To configure basic routing setup, use these components: BrowserRouter, Routes, Route, Link

- BrowserRouter enables routing functionality across the app.
- Routes holds each route.
- Route maps each URL path to a component.
- Link generates navigational links that change the view without reloading.

In the basic setup in the next page ➠, `BrowserRouter` wraps the entire application, which allows React Router to manage the routing. We define the `Routes component`, which holds multiple Route elements. Each Route specifies a path (like / for the home page) and an element (like `<Home />` or `<About />`) that gets rendered when the path is matched. This approach is ideal for quick setups and demos, where you might not need to split everything into separate files.

PS: You most likely won't do this while building your applications.

See the basic setup with Routes ➠



Click to load iframe

https://codesandbox.io/embed/vq9qn3?view=editor+%2B+preview&module=%2Fsrc%2FApp.js%3A12%2C17

In a more structured setup, each route's component (e.g., Home, About, and NavBar) is imported from its own file. This setup keeps App.js `cleaner and allows for better organization`, especially as your application grows. As you would see below, index.js wraps the App component in BrowserRouter, enabling client-side routing while App.js defines routes and imports components for each path.



Click to load iframe

https://codesandbox.io/embed/nlcw6p?view=editor+%2B+preview&module=%2Fsrc%2FApp.js%3A11%2C1

Navigating Between Pages

Navigation in React Router is handled by the Link and useNavigate components, allowing users to navigate both via `clicks` and `programmatically`.

✴ Client-Side Navigation with Link: Using "Link" components replaces traditional anchor tags, allowing for smooth client-side transitions.

```
import { Link } from 'react-router'

export default function NavigationBar() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
    </nav>
  )
}
```

✴ Programmatic Navigation with useNavigate: To navigate in response to an event (like a form submission), use the useNavigate hook.

```
import { useNavigate } from 'react-router'

function LoginPage() {
  const navigate = useNavigate()

  const handleLogin = () => {
    // Login logic
    navigate('/dashboard')
  }

  return <button onClick={handleLogin}>Login</button>
}
```

Dynamic Routing and URL Parameters

Dynamic routing in React Router enables `a single route to handle multiple URL variations` by using URL parameters. This is especially useful when building pages with variable content, such as user profiles, product details, or blog posts, where each view depends on a unique identifier like an ID or a username.

For example, a route like /profile/:username can dynamically render different content based on the `username parameter` in the URL.

In the example in the next page ➠, here is what happens:

1 When the app loads, it displays the Home component. The user can `enter a username` and click "Go to Profile."

2 The handleSubmit function in Home.js `uses navigate to redirect` the user to `/profile/:username`, replacing with the actual username entered.

3 The Profile component `extracts the username from the URL using useParams`, displaying a personalized message.



Click to load iframe

https://codesandbox.io/embed/nqzq6v?view=preview&module=%2Fsrc%2FProfile.js

Redirects and Protected Routes

Redirects and protected routes are essential for `securing certain parts` of your application or `guiding users` to appropriate pages. Protected routes, for instance, restrict access to authenticated users, while redirects can send users to a different path if certain conditions aren't met.

In the next example in the next page ➠, we create a protected route that only allows authenticated users to access it. If the user is not authenticated, they are redirected to the login page.

1 The ProtectedRoute component checks if the user is authenticated.

2 If isAuthenticated is false, it redirects to /login using the component, effectively preventing access to the protected Dashboard route.

3 If isAuthenticated is true, it renders the child component (in this case, Dashboard), granting access.



Click to load iframe

https://codesandbox.io/embed/vhts7t?view=preview&module=%2Fsrc%2FProtectedRoute.js

React Router vs. Tanstack Router

React Router is a powerful library for managing routing in React applications, but it can be complex and sometimes overwhelming. Tanstack Router is a newer alternative that aims to simplify routing while providing similar functionality. It offers a more intuitive API, better performance, and improved developer experience.

Tanstack Router is designed to be more flexible and easier to use, making it a great choice for developers looking for a simpler routing solution. It also integrates well with other Tanstack libraries, such as Tanstack Query, providing a cohesive experience for managing data and routing in your applications.

Key Differences

Feature	React Router	Tanstack Router
API Complexity	More complex, with a steeper learning curve	Simpler and more intuitive API
Performance	Good, but can be slower with large apps	Optimized for better performance
Data Fetching	Requires additional libraries (e.g., React Query)	Built-in data fetching capabilities
Integration	Integrates with various libraries	Seamless integration with Tanstack libraries
Community	Large and active community	Growing community, but smaller than React Router
Documentation	Comprehensive, but can be overwhelming	Clear and concise documentation
Learning Resources	Abundant tutorials and resources	Fewer resources available
Flexibility	Highly flexible, but can lead to complexity	More opinionated, but easier to use
State Management	Requires external libraries (e.g., Redux)	Built-in state management capabilities

TanstackRouter

Tanstack Router is a powerful and flexible routing library for React applications. It provides a simple and intuitive API for managing routes, making it easy to create complex navigation structures. With Tanstack Router, you can define routes, handle URL parameters, and manage nested routes with ease.

Step 1: install the router

```
# bun
bun add @tanstack/react-router @tanstack/react-router-devtools
bun add -D @tanstack/router-plugin
```

Step 2: Configure Vite Plugin

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import { tanstackRouter } from '@tanstack/router-plugin/vite'

export default defineConfig({
  plugins: [
    tanstackRouter({ target: 'react', autoCodeSplitting: true }),
    react(),
  ],
})
```

Step 3: Create Basic Router Setup

```
// Render the app
const rootElement = document.getElementById('app')!
if (!rootElement.innerHTML) {
  const root = ReactDOM.createRoot(rootElement)
  root.render(
    <StrictMode>
      <RouterProvider router={router} />
    </StrictMode>,
  )
}
```



Click to load iframe

https://stackblitz.com/edit/vitejs-vite-nfqyhumz?ctl=1&embed=1&file=src%2FApp.jsx

- 100% TypeScript Support - Fully type-safe routing
- File-based Routing - Automatic route generation
- Nested Routing - Complex route hierarchies
- Search Params - Built-in URL state management
- Data Loading - Route-level data fetching
- DevTools - Built-in development tools
- Code Splitting - Automatic bundle optimization

```
# Create new project
npx create-tsrouter-app@latest my-app --template file-router

# Navigate and install
cd my-app
npm install
npm run dev
```

The support for file based routing, route file generation with the route gen tree is truly amazing and innovative.



Click to load iframe

https://stackblitz.com/edit/tanstack-router-k5akabnz?ctl=1&embed=1&view=preview

Frameworks

Frameworks are pre-built libraries that provide a structure for building applications. They often include built-in features like routing, state management, and data fetching, making it easier to develop complex applications quickly. Some popular frameworks include:

- **Next.js**: A React framework for building server-rendered applications. It provides features like static site generation, API routes, and built-in CSS support.
- **Tanstack Start**: A new framework for building web applications using Tanstack Router based and good principles of tanstack query. It provides a simple and intuitive API for managing routes, making it easy to create complex navigation structures.
- **Remix** and **React Router v7** : A full-stack React framework that focuses on server-rendered applications. It provides features like data loading, nested routes, and built-in caching.
- **Redwood SDK**: is a React framework for building server-side web apps on Cloudflare
- **Waku**, **Vike**

Next.js

Next.js is a React framework that enables server-side rendering and static site generation. It provides a powerful routing system, allowing you to create dynamic routes and handle URL parameters easily. Next.js also offers features like API routes, built-in CSS support, and automatic code splitting, making it a great choice for building modern web applications. With its App Router and Pages Router, Next.js allows you to choose between a file-based routing system and a more flexible approach, giving you the best of both worlds. It has the pages and app router system. Old apps using pages is fine but new project should come in app router.

- Simple Auth in Next(pages router)
- Sample Todo App - Live URL

Tanstack Start

Tanstack Start is a new framework for building web applications using Tanstack Router. It provides a simple and intuitive API for managing routes, making it easy to create complex navigation structures. With Tanstack Start, you can define routes, handle URL parameters, and manage nested routes with ease. It also integrates well with other Tanstack libraries, such as Tanstack Query, providing a cohesive experience for managing data and routing in your applications.

New example project can be created using Tanstack Start CLI(npm create @tanstack/start@latest), Tanstack Builder(pnpx create-start-app@latest)

SEO

Unhead has first-class support for React, allowing you to manage your head tags using a `<Head>` component, the `useHead()` hook, and other ecosystem hooks.

It can directly replace react-helmet, handling a more diverse set of use cases from SEO to structured data.

- Install the `@unhead/react` package
- Setup the provider using `const head = createHead()`
- `UnheadProvider` will wrap your application with the `head` prop.

```
import { createHead, UnheadProvider } from '@unhead/react/client'

const head = createHead()

const root = createRoot( document.getElementById('root'));

root.render(
  <StrictMode>
    <UnheadProvider head={head}>
      <App />
    </UnheadProvider>
  </StrictMode>,
)
```

Assignment

Build a simple `Blog Application` with `React Router` and `Tanstack Router`. The app should have a `Home Page` displaying a list of blog posts(fetch the post from the API). Each post should have a title and a brief excerpt, with a link to its `Detail Page`. The `Detail Page` should show the full content of the selected blog post, which can be dynamically generated. Users should be able to go back to the `Home Page` from any page or the previous page. Also, handle 404 errors by displaying a fallback page when a non-existent route is visited. Implement Error Boundary and Suspense with LazyLoading.

Try out Tanstack Router for a different experience.

We will discuss more about frameworks in the next semester. Nextjs, ReactRouterv7, Tanstack, and more.

State Management context API, redux

Best Practices

State Management in React

- Lifting State Up in React
- Context API for Global State
- Introduction to Redux
- Best Practices for State Management
- Activity and ViewTransition

Lifting State Up in React

State Management in  React is one of the beauty of the library that makes it shine. When a user *interacts* with a reactive element or component, an update which is simply known as *change in state* occur, instantly updating the UI or performing a defined task.

In most cases, we perform a "State Update" within the component where it is happening. But how about when we want other components in the same *parent* component to be aware of the update or interaction? This is where Lifting Up State comes in handy, which involves moving the state management that is occurring in "Child1" component to the "Parent" component so that the "Child2" or other children of the parent, including the parent itself would be aware of that particular *State Change*.

Lifting Up State in  React can simply be defined as the process of moving state to the nearest common ancestor of components that need to share it.

Now that we understand the definition and purpose of Lifting Up State, lets look at it's code structure in the next slide.

Lifting Up State in  React Code Structure

The structure of a *Lifted State* is just as its name sounds; Lifting Up.

```
function Parent() {
  const [value, setValue] = useState('')

  return (
    <div>
      <ChildA value={value} />
      <ChildB setValue={setValue} />
    </div>
  )
}

function ChildA({ value }) {
  return <p>{value}</p>
}

function ChildB({ setValue }) {
  return <input onChange={(e) => setValue(e.target.value)} />
}
```

In the code example above, we see a transition where we were previously managing state in "ChildA" and then making it available everywhere by *Lifting Up The State* to the parent.

Lifting Up State in  React Code Example in action

Here is the interactive mode of the code example in the previous slide.



Click to load iframe

https://codesandbox.io/embed/pzvrd4

Context API for Global State

There are cases where we would want to share a state across the entire application or to one or more parents in different folders of our application. In this case, simply Lifting Up State is just not enough and won't really be clean code if used because you would have to pass the state as props to any child component that needs it. Example of this complex case is *Theme state management* where would want our entire application to be aware of which theme was toggled. This is where React's *Context API* comes in handy.

Context API is a built-in React feature to create and manage global state without passing props manually.

How It Works:

- Create a context using `React.createContext`.
- Wrap your application in a `Provider`.
- Use `useContext` or `use` to consume the context in child components.

Usage of Context API

Below is a code example of how the Context API is used.

```
const MyContext = React.createContext()

function App() {
  const [value, setValue] = useState('Hello')

  return (
    <MyContext.Provider value={{ value, setValue }}>
      <Child />
    </MyContext.Provider>
  )
}

function Child() {
  // const { value, setValue } = use(MyContext)
  const { value, setValue } = useContext(MyContext)
  return <button onClick={() => setValue('Updated')}>{value}</button>
}
```

Context API Code Example in action

Here is a theme context example code in action.



Click to load iframe

https://codesandbox.io/embed/kwslzk

Introduction to Redux

For large-scale applications with complex state interactions, we can use external library like Redux for state management and Redux Toolkit(RTK) for more flexibility.

Redux is a state management library for predictable state management in large applications.

Core Concepts:

- Store: Holds the global state.
- Actions: Plain objects that describe what you want to do.
- Reducers: Pure functions that update the state based on actions.
- Dispatch: Triggers an action.
- Redux Toolkit: Managing Async State

```
// actions.js
export const increment = () => ({ type: 'INCREMENT' })

// reducer.js
const initialState = { count: 0 }
export const counterReducer = (state = initialState, action) => {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, count: state.count + 1 }
    default:
      return state
  }
}

// store.js
import { createStore } from 'redux'
const store = createStore(counterReducer)
```

More Thirty Party Libraries for State management includes `zustand`, `jotai`, `@tanstack/query`, `swr`, `recoil`, `xstate`.

```tsx
import { useQuery } from '@tanstack/react-query'

function UserProfile({ userId }: { userId: string }) {
  const { data: user, isLoading, error } = useQuery({
    queryKey: ['user', userId],
    queryFn: () => fetchUser(userId),
  })

  if (isLoading) return <div>Loading...</div>
  if (error) return <div>Error loading user</div>

  return <div>{user?.name}</div>
}
```

Best Practices for State Management

State management best practices ensure your application's state is scalable, maintainable, and performant.

Below are best practices that matter in key areas:

1. Scalability:
   - Use local state for component-specific needs and global state (e.g., `Redux`, `Context API`) for cross-component data.
   - Split state into smaller, logical chunks (e.g., authentication, UI state, user data) to avoid a "giant state object."
2. Performance Optimization
   - Keep state minimal and colocate it where it's used.
   - Avoid prop drilling by using `Context API` or libraries like `Redux` or `Zustand`.
   - Use memoization techniques (`React.memo`, `useMemo`, `useCallback`) to prevent re-rendering of unaffected components.
3. Maintainability
   - Write clean reducers (in `Redux`) or handlers for state updates.
   - Use type safety (e.g., `TypeScript`) to catch state-related errors at compile time.
   - Document state structure and logic for better team collaboration.
4. Debugging and Predictability
   - Use DevTools (e.g., Redux DevTools, React Developer Tools) for tracking state changes.
   - Ensure state updates are pure functions (no side effects).
5. User Experience
   - Synchronize state changes with UI updates (e.g., loading indicators, error handling).
   - Persist critical state (e.g., user sessions) using `localStorage` or `sessionStorage` when necessary.
6. Avoid Over-Engineering
   - Use React local state or `Context API` for simple use cases.
   - Only introduce libraries like `Redux` when the state becomes too complex for `Context API`.
7. Reusability and Composition
   - Use `custom hooks` for common state logic (e.g., `useAuth`, `useTheme`).
   - Group related state in composable chunks for better separation of concerns.

Activity and View Transition

Activity Component (React 19.2)

- `<Activity>` hides children via `display: none` while preserving their state and DOM. Stateful hidden components.
- `mode="visible" | "hidden"` controls whether content shows; hidden content still re-renders at lower priority.
- Effects inside children are cleaned up while hidden; restored when visible again.
- Ideal for tabs/sidebars you want to hide without losing state or refetching or prefetching(Media like videos).

```
1  import { Activity, useState } from 'react'
2
3  function App() {
4    const [showSidebar, setShowSidebar] = useState(true)
5
6    return (
7      <div className="layout">
8        <button onClick={() => setShowSidebar(prev => !prev)}>
9          Toggle sidebar
10       </button>
11
12       <Activity mode={showSidebar ? 'visible' : 'hidden'}>
```

Note: Hidden video/audio/iframe elements keep playing unless their own cleanup stops them.

View Transition

- The View Transitions API enables animated DOM swaps for route or view changes. Read More about SPA use-cases
- Wrap DOM updates in `document.startViewTransition` when available; it falls back silently if unsupported.
- Use CSS to define shared element animations (`view-transition-name`).
- You can wrap pages, component or group interaction with element and setup a minimal style.

```
::view-transition-group(root){
  animation-duration: 200ms;
}

::view-transition-old(root) {
  animation-name: slide-out-to-left;
}

::view-transition-new(root) {
  animation-name: slide-in-from-right;
}
```

Data Fetching fetch, axios, react query

Working with APIs

Data Fetching and Working with APIs

- Fetching Data with "fetch" and "axios"
- Handling Asynchronous Data and Promises
- Error Handling and Loading States
- Using SWR for Data Fetching
- Using React Query for Data Management
- Suspense + Lazy Loading + Error Boundaries
- SSR and RSC

Fetching Data with "fetch" and "axios"

Data Fetching is one of the key aspect of coding. Often times, we would want to fetch data from an external library or database by consuming APIs connecting us to the backend where the data are stored. There would also be a need where you want to send to the database instead, or update it. This process of sending and/or receiving data from an external or backend database is termed as *Data Fetching*.

How do we perform Data Fetching in  React?

We can achieve this using the built-in `fetch()` API, or by using a third-party library like `Axios()`

- fetch() API: A built-in JavaScript API for making HTTP requests. It returns a Promise and works natively in modern browsers. While lightweight, it requires manual handling of JSON parsing (`response.json()`) and HTTP errors (`response.ok`).

- axios: A third-party library that simplifies HTTP requests. It has built-in support for JSON transformation, better error handling, and allows configuration of global defaults (e.g., base URLs or headers). For instance, `axios.get('/api/data')` is simpler than setting up options for fetch.

Using the fetch() API

To use the `fetch()` API in React, you need to properly manage state to avoid infinite API calls and simply call the `fetch()` in a promise or an async/await function, passing the methods, header, authorization, etc, if needed. Below are simply performing a `GET` request without error handling which is addressed in subsequent slides. If you will use `useEffect` it should be done like this but it is generally discouraged in favour of `swr` or `@tanstack/react-query`.

```
1  const FetchDataExample = () => {
2    const [data, setData] = React.useState(null)
3
4    React.useEffect(() => {
5      let ignore = false;
6
7      const fetchData = async () => {
8        const response = await fetch('https://api.oluwasetemi.dev/posts')
9
10       if (!response.ok) console.error('Network response was not ok')
11
12       const jsonData = await response.json()
13       setData(jsonData)
14     }
15
16     if (!ignore) {
17       fetchData()
18     }
19
```

Using Axios

Since `Axios` is a third-party library, you would need to install the package by running `npm install axios`. Once it is installed, import the package to your component and properly manage state to avoid infinite requests. Again, error handling is detailed in the subsequent slide on properly handling errors while fetching data.

```
import React, { useEffect, useState } from 'react'
import axios from 'axios'

const AxiosDataExample = () => {
  const [data, setData] = useState(null)

  useEffect(() => {
    let ignore = false;

    axios
      .get('https://api.oluwasetemi.dev/posts')
      .then((response) => {
          if (!ignore) {
            setData(response.data)
          }
        })

    return () => {
      ignore = true
```

Handling Asynchronous Data and Promise

In the previous slides a hint of asynchronous fetching was seen. Let's talk more on how to properly handle asynchronous fetching.

Steps:

- Define a function to handle the asynchronous call.
- Use `async/await` for better readability or `.then()` for chaining promises.
- Manage errors with `try/catch` or `.catch()`.

```
// Using .then()
useEffect(() => {
  let mounted = true;

  fetch('https://api.oluwasetemi.dev/posts')
    .then((response) => {
      if (!response.ok) throw new Error('Network response was not ok')
      return response.json()
    })
    .then((jsonData) => { if (mounted) setData(jsonData) })
    .catch((err) => { if (mounted) setError(err.message) })

  return () => {
    mounted = false;
```

Error Handling and Loading States

React applications should provide a seamless user experience when loading or when errors occur.

This involves:

- Loading States: Displaying a loading spinner or message until data is fetched.
- Error Handling: Showing user-friendly error messages when a request fails.
- Data: Set when the data is successfully retrieved.

```
const [data, setData] = useState(null)
const [loading, setLoading] = useState(true)
const [error, setError] = useState(null)

useEffect(() => {
  let ignore = false;

  const fetchData = async () => {
    setLoading(true) // Start loading
    try {
      const response = await fetch('https://api.oluwasetemi.dev/posts')
      if (!response.ok) throw new Error('Failed to fetch data')
      const jsonData = await response.json()
      setData(jsonData)
    } catch (err) {
      setError(err.message)
    } finally {
```

Using SWR for Data Fetching

SWR is a lightweight data-fetching library from Vercel. It focuses on caching, revalidation, and subscription-like updates (focus/reconnect/interval) with a tiny API. Check the docs.

- Install: `npm/pnpm/bun install swr`
- Core hook: `useSWR(key, fetcher, options)` for GET + revalidation
- Mutations: `useSWRMutation(key, mutator, { onSuccess: () => mutate() })`
- Subscription-style updates: `revalidateOnFocus`, `revalidateOnReconnect`, `refreshInterval`, and manual `mutate`. `useSWRSubscription` - is a React hook that allows subscribing to real-time data sources with SWR.
- You can enable the suspense option to use SWR with React Suspense. `{suspense: true}`
- Pagination support(normal and infinite loading) and Prefetching of Data.

```
import useSWR from 'swr'
import useSWRMutation from 'swr/mutation'

const fetcher = async (url) => {
  const res = await fetch(url)
  if (!res.ok) throw new Error('Failed to fetch')
  return res.json()
}

const postComment = async (url, { arg }) => {
  const res = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(arg),
  })
  if (!res.ok) throw new Error('Failed to mutate')
  return res.json()
}

function Comments() {
  const { data, error, isLoading, mutate } = useSWR(
    'https://api.oluwasetemi.dev/comments',
    fetcher,
```

Using React Query for Data Management

React Query simplifies server-state management with caching, background updates, and easy loading/error handling. It is a powerful way of managing states in a  React app.

This involves:

- Install React Query: `npm install @tanstack/react-query`
- Set up a `QueryClient` and `QueryClientProvider` in your app.
- Use the `useQuery` hook to fetch and manage data.

```
1  import { useQuery, QueryClient, QueryClientProvider, } from '@tanstack/react-query'
2
3  const queryClient = new QueryClient()
4
5  const fetchPosts = async () => {
6    const response = await fetch('https://api.oluwasetemi.dev/posts')
7    if (!response.ok) throw new Error('Network response was not ok')
8    return response.json()
9  }
10
11 const ReactQueryExample = () => {
12   const { data, error, isLoading } = useQuery(['posts'], fetchPosts)
13
14   if (isLoading) return <p>Loading...</p>
15   if (error) return <p>Error: {error.message}</p>
16
```



Click to load iframe

https://stackblitz.com/edit/github-zvtcsu?ctl=1&embed=1&file=src%2Findex.tsx&hideExplorer=1

Suspense for Data Fetching

Suspense lets you declaratively specify the loading state for a part of your component tree while waiting for async operations (data loading, code splitting, etc.).

Key Benefits:

- Declarative loading states - no manual `isLoading` flags
- Coordinates multiple async operations
- Prevents "waterfall" loading patterns
- Works with lazy imports and data fetching libraries

Syntax:

```
<Suspense fallback={<Loading />}>
  <ComponentThatSuspends />
</Suspense>
```

What can trigger Suspense?

- Data fetching with Suspense-enabled frameworks (React Query with `useSuspenseQuery`, SWR with `{suspense: true}`)
- Lazy-loaded components with `React.lazy()`
- Reading from Suspense-enabled resources

Suspense Example with React Query

```
1  import { Suspense } from 'react'
2  import { useSuspenseQuery } from '@tanstack/react-query'
3
4  const fetchUser = async () => {
5    const res = await fetch('https://api.oluwasetemi.dev/user/1')
6    if (!res.ok) throw new Error('Failed to fetch')
7    return res.json()
8  }
9
10 function UserProfile() {
11   // useSuspenseQuery automatically suspends during loading
12   const { data } = useSuspenseQuery({
13     queryKey: ['user'],
14     queryFn: fetchUser,
15   })
16
17   return <div>Welcome, {data.name}!</div>
18 }
19
20 function App() {
21   return (
```

Docs: React Suspense

Lazy Loading with Suspense

Code splitting reduces initial bundle size by loading components on demand.

Without Lazy Loading:

```
import HeavyComponent from './HeavyComponent'
function App() {
  return (
    <div><HeavyComponent /></div>
  )
}
```

Entire component loaded upfront

Route-based code splitting:

```
const Home = lazy(() => import('./pages/Home'))
const About = lazy(() => import('./pages/About'))
const Contact = lazy(() => import('./pages/Contact'))

<Suspense fallback={<PageLoader />}>
```

With Lazy Loading:

```
import { lazy, Suspense } from 'react'

const HeavyComponent = lazy(() =>
  import('./HeavyComponent')
)

function App() {
  return (
    <Suspense fallback={<div>Loading ... </div>}>
      <HeavyComponent />
    </Suspense>
  )
}
```

Component loaded on demand

Error Boundaries

Error Boundaries catch JavaScript errors anywhere in the component tree, log them, and display a fallback UI.

What Error Boundaries catch:

- Errors during rendering
- Errors in lifecycle methods
- Errors in constructors of components below them

What they DON'T catch:

- Event handlers (use try/catch instead)
- Async code (setTimeout, promises)
- Server-side rendering errors
- Errors in the error boundary itself

Class Component Implementation:

```
class ErrorBoundary extends React.Component {
  state = { hasError: false }

  static getDerivedStateFromError(error) {
    return { hasError: true }
  }

  componentDidCatch(error, errorInfo) {
    console.error('Error caught:', error, errorInfo)
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>
    }
    return this.props.children
  }
}
```

react-error-boundary Library

The `react-error-boundary` library provides a simpler, hook-based way to handle errors.

Installation:

```
function App() {
  return (
    <ErrorBoundary
      FallbackComponent={ErrorFallback}
      onReset={() => {
        // Reset app state
      }}
      onError={(error, errorInfo) => {
        // Log to error reporting service
        console.error('Logged error:', error, errorInfo)
      }}
    >
      <MyComponent />
    </ErrorBoundary>
  )
}
```

Docs: react-error-boundary

Combining Suspense + Error Boundaries

Best practice: Wrap Suspense boundaries with Error Boundaries for robust error handling.

```
import { Suspense, lazy } from 'react'
import { ErrorBoundary } from 'react-error-boundary'
import { useSuspenseQuery } from '@tanstack/react-query'

const Dashboard = lazy(() => import('./Dashboard'))

function ErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div className="error">
      <h2>Oops! Something went wrong</h2> <p>{error.message}</p>
      <button onClick={resetErrorBoundary}>Retry</button>
    </div>
  )
}
function App() {
  return (
    <ErrorBoundary FallbackComponent={ErrorFallback}>
      <Suspense fallback={<div>Loading dashboard...</div>}>
        <Dashboard />
      </Suspense>
    </ErrorBoundary>
```

Pattern: Error Boundary → Suspense → Async Component

Server-Side Rendering (SSR) APIs

React provides APIs for rendering components on the server for improved performance and SEO.

Legacy APIs (Node.js Streams):

- `renderToString()` - Renders tree to HTML string (blocks until complete)
- `renderToStaticMarkup()` - Like renderToString but without React attributes
- `renderToNodeStream()` - Renders to Node.js readable stream (deprecated)
- `renderToStaticNodeStream()` - Static version of renderToNodeStream (deprecated)

Modern APIs (Web Streams):

- `renderToReadableStream()` - Renders to Web ReadableStream (supports Suspense). `resume` resumes `prerender` to a Readable Web Stream.
- `renderToPipeableStream()` - Renders to pipeable Node.js stream (supports Suspense & streaming)

Client Hydration APIs:

- `hydrateRoot()` - Attaches React to server-rendered HTML
- `createRoot()` - Creates root for client-only rendering

SSR Example with renderToString